

DATA-DRIVEN PREDICTION OF BATTERY FAILURE IN ELECTRIC VEHICLES

*Report Submitted in partial fulfillment of requirements for the
B.Tech. degree in Electrical Engineering*

BY

NAME	ROLL NUMBER
Manvi Gupta	2020UEE4509
Archie Arya	2020UEE4518
Uday Varshney	2020UEE4529
Jishnu Mittal	2020UEE4542

**Under the supervision
of
Dr. Anuradha Tomar**



**Department of Electrical Engineering
Netaji Subhas University of Technology (NSUT) New Delhi
India-110078
MAY 2024**

CANDIDATE(S) DECLARATION



DEPARTMENT OF ELECTRICAL ENGINEERING

We, **Manvi Gupta (2020UEE4509)**, **Archie Arya (2020UEE4518)**, **Uday Varshney (2020UEE4529)**, and **Jishnu Mittal (2020UEE4542)** of B.Tech. Department of Electrical Engineering, hereby declare that the Project-Thesis titled ***“Data-Driven Prediction of Battery Failure in Electric Vehicles”*** which is submitted by us to the Department of Electrical Engineering, Netaji Subhas University of Technology (NSUT) Dwarka, New Delhi in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology is original and not copied from the source without proper citation. The manuscript has been subjected to plagiarism checks by Turnitin software. This work has not previously formed the basis for the award of any Degree.

Place :

Date :

Manvi Gupta
2020UEE4509

Archie Arya
2020UEE4518

Uday Varshney
2020UEE4529

Jishnu Mittal
2020UEE4542

CERTIFICATE OF DECLARATION



DEPARTMENT OF ELECTRICAL ENGINEERING

This is to certify that the work embodied in project thesis titled, ***“Data-Driven Prediction of Battery Failure in Electric Vehicles”*** by **Manvi Gupta (2020UEE4509)**, **Archie Arya (2020UEE4518)**, **Uday Varshney (2020UEE4529)**, and **Jishnu Mittal (2020UEE4542)** is the bonafide work of the group submitted to Netaji Subhas University of Technology for consideration in 8th Semester B.Tech. Project Evaluation. The original Research work was carried out by the team under my guidance and supervision in the academic year 2022-2023. This work has not been submitted for any other diploma or degree of any university. On the basis of a declaration made by the group, we recommend the project report for evaluation.

Dr. Anuradha Tomar

(Assistant Professor)

Dept. of Instrumentation & Control Engineering

Netaji Subhas University of Technology

ACKNOWLEDGEMENT

We would like to express my gratitude and appreciation to all those who make it possible to complete this project. Special thanks to our project supervisor Dr. Anuradha Tomar whose help, stimulating suggestions and encouragement helped us in writing this report. We also sincerely thank our colleagues for the time spent proofreading and correcting our mistakes.

We would also like to acknowledge with much appreciation the crucial role of the staff in Electrical Engineering, who gave us permission to use the lab and the systems and gave permission to use all necessary things related to the project.

Manvi Gupta

(2020UEE4509)

Archie Arya

(2020UEE4518)

Uday Varshney

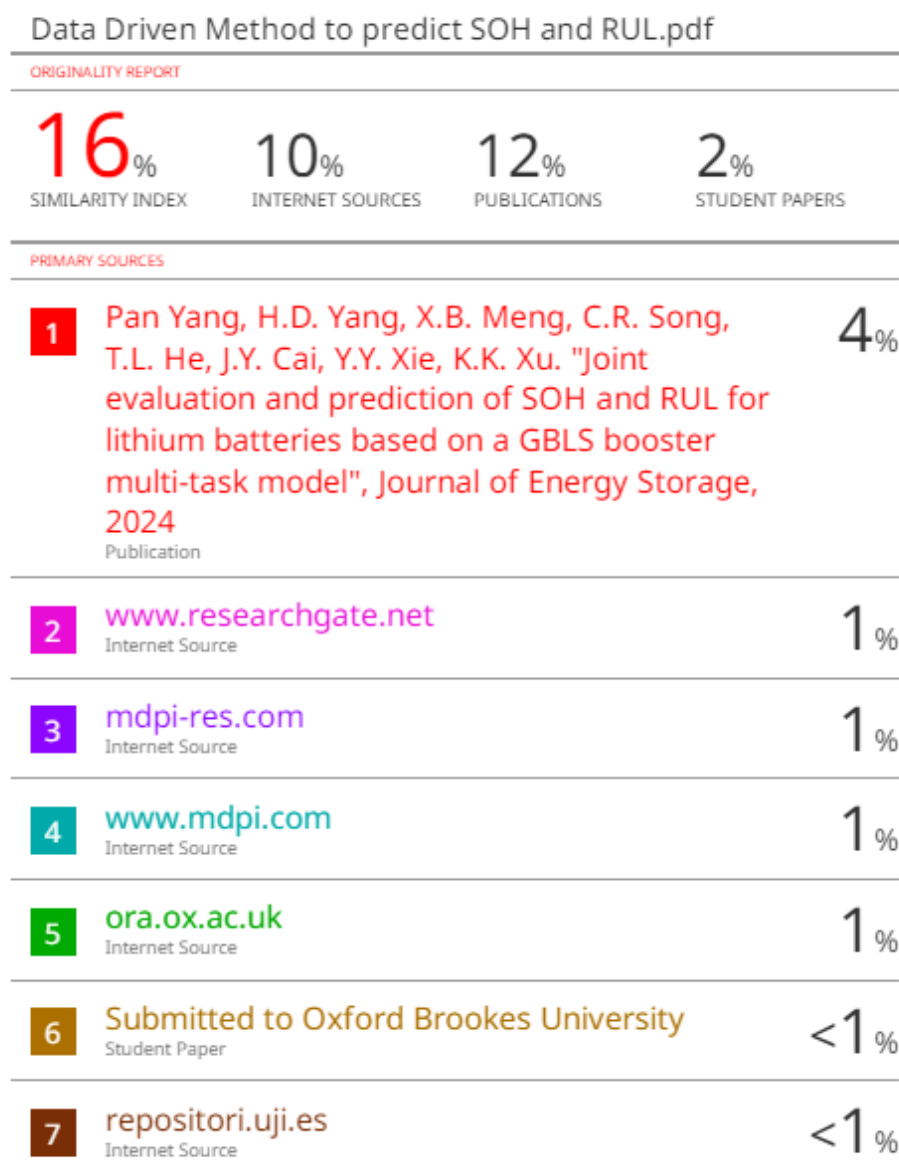
(2020UEE4529)

Jishnu Mittal

(2020UEE4542)

PLAGIARISM REPORT

Attached is the plagiarism report for our paper and the presented report.



8	Radhika Swarnkar, Harikrishnan Ramachandran, Sawal Hamid Md Ali, Rani Jabbar. "A Systematic Literature Review of State of Health and State of Charge Estimation Methods for Batteries Used in Electric Vehicle Applications", World Electric Vehicle Journal, 2023 Publication	<1 %
9	Zihao Zhou, Aihua Ran, Shuxiao Chen, Xuan Zhang, Guodan Wei, Baohua Li, Feiyu Kang, Xiang Zhou, Hongbin Sun. "A fast screening framework for second-life batteries based on an improved bisecting K-means algorithm combined with fast pulse test", Journal of Energy Storage, 2020 Publication	<1 %
10	5dok.net Internet Source	<1 %
11	He, Zhiwei, Mingyu Gao, Guojin Ma, Yuanyuan Liu, and Sanxin Chen. "Online state-of-health estimation of lithium-ion batteries using Dynamic Bayesian Networks", Journal of Power Sources, 2014. Publication	<1 %
12	arxiv.org Internet Source	<1 %
13	digitalcommons.unl.edu Internet Source	

		<1 %
14	ojs.jurnaltechne.org Internet Source	<1 %
15	Yong Qiu, Xu Zhang, Yun Tian, Zhen Zhou. "Machine learning promotes the development of all-solid-state batteries", Chinese Journal of Structural Chemistry, 2023 Publication	<1 %
16	Wenwen Zhang, Jianfang Jia, Xiaoqiong Pang, Jie Wen, Yuanhao Shi, Jianchao Zeng. "An Improved Transformer Model for Remaining Useful Life Prediction of Lithium-Ion Batteries under Random Charging and Discharging", Electronics, 2024 Publication	<1 %
17	Xudong Qu, Dapai Shi, Jingyuan Zhao, Manh- Kien Tran, Zhenghong Wang, Michael Fowler, Yubo Lian, Andrew F. Burke. "Insights and reviews on battery lifetime prediction from research to practice", Journal of Energy Chemistry, 2024 Publication	<1 %
18	Yingjian Zhuge, Hengzhao Yang, Haoyu Wang. "Overview of Machine Learning- Enabled Battery State Estimation Methods",	<1 %

2023 IEEE Applied Power Electronics
Conference and Exposition (APEC), 2023
Publication

19	assets.researchsquare.com Internet Source	<1 %
20	pubmed.ncbi.nlm.nih.gov Internet Source	<1 %
21	www.process.st Internet Source	<1 %
22	Ramy M. Khalifa, Soumaya Yacout, Samuel Bassetto. "Developing machine-learning regression model with Logical Analysis of Data (LAD)", Computers & Industrial Engineering, 2021 Publication	<1 %
23	Brian Ospina Agudelo, Walter Zamboni, Fabio Postiglione, Eric Monmasson. "Battery State-of-Health estimation based on multiple charge and discharge features", Energy, 2022 Publication	<1 %
24	Zuolu Wang, Guojin Feng, Dong Zhen, Fengshou Gu, Andrew D. Ball. "Chapter 57 State of Health Estimation of Lithium-Ion Batteries from Charging Data: A Machine Learning Method", Springer Science and Business Media LLC, 2023 Publication	<1 %

25	bura.brunel.ac.uk Internet Source	<1 %
26	luminagold.com Internet Source	<1 %
27	siliconcanals.com Internet Source	<1 %
28	www.epdoc.cn Internet Source	<1 %
29	www.sciencegate.app Internet Source	<1 %
30	Submitted to University of Greenwich Student Paper	<1 %
31	export.arxiv.org Internet Source	<1 %
32	repository.tudelft.nl Internet Source	<1 %
33	www.grin.com Internet Source	<1 %
34	www.spiedigitallibrary.org Internet Source	<1 %
35	Mingyu Gao, Zhengyi Bao, Chunxiang Zhu, Jiahao Jiang, Zhiwei He, Zhekang Dong, Yining Song. "HFCM-LSTM: A novel hybrid	<1 %

framework for state-of-health estimation of lithium-ion battery", Energy Reports, 2023

Publication

-
- | | | |
|-----------|--|----------------|
| 36 | Yaxiang Fan, Fei Xiao, Chaoran Li, Guorun Yang, Xin Tang. "A novel deep learning framework for state of health estimation of lithium-ion battery", Journal of Energy Storage, 2020 | <1 % |
|-----------|--|----------------|

Publication

-
- | | | |
|-----------|---|----------------|
| 37 | assets.publishing.service.gov.uk | <1 % |
|-----------|---|----------------|

Internet Source

-
- | | | |
|-----------|---|----------------|
| 38 | kipdf.com | <1 % |
|-----------|---|----------------|

Internet Source

-
- | | | |
|-----------|---|----------------|
| 39 | Florin Leon, Silvia Curteanu, Cătălin Lisa, Nicolae Hurduc. "Machine Learning Methods Used to Predict the Liquid-Crystalline Behavior of Some Copolyethers", Molecular Crystals and Liquid Crystals, 2007 | <1 % |
|-----------|---|----------------|

Publication

-
- | | | |
|-----------|--|----------------|
| 40 | Haokai Ruan, Hongwen He, Zhongbao Wei, Zhongyi Quan, Yunwei Li. "State of Health Estimation of Lithium-ion Battery Based on Constant-Voltage Charging Reconstruction", IEEE Journal of Emerging and Selected Topics in Power Electronics, 2021 | <1 % |
|-----------|--|----------------|

Publication

41	M.S. Reza, M. Mannan, M. Mansor, Pin Jern Ker, T. M. Indra Mahlia, M.A. Hannan. "Recent advancement of remaining useful life prediction of lithium-ion battery in electric vehicle applications: A review of modelling mechanisms, network configurations, factors, and outstanding issues", Energy Reports, 2024 Publication	<1 %
42	Xin Xiong, Yujie Wang, Cong Jiang, Xingchen Zhang, Haoxiang Xiang, Zonghai Chen. "End-to-end deep learning powered battery state of health estimation considering multi-neighboring incomplete charging data", Energy, 2024 Publication	<1 %
43	bspace.buid.ac.ae Internet Source	<1 %
44	pangea.stanford.edu Internet Source	<1 %
45	purehost.bath.ac.uk Internet Source	<1 %
46	researchonline.ljmu.ac.uk Internet Source	<1 %
47	www.ros.hw.ac.uk Internet Source	<1 %

48	www2.mdpi.com Internet Source	<1 %
49	Dongxu Shen, Lifeng Wu, Guoqing Kang, Yong Guan, Zhen Peng. "A novel online method for predicting the remaining useful life of lithium-ion batteries considering random variable discharge current", <i>Energy</i> , 2021 Publication	<1 %
50	Jiwei Yao, John D. Hedengren, Tao Gao, Kody M. Powell. "A two-level optimization framework for battery energy storage systems to enhance economics and minimize long-term capacity fading", <i>Journal of Energy Storage</i> , 2023 Publication	<1 %
51	Yadyra Ortiz, Paul Arévalo, Diego Peña, Francisco Jurado. "Recent Advances in Thermal Management Strategies for Lithium-Ion Batteries: A Comprehensive Review", <i>Batteries</i> , 2024 Publication	<1 %
52	Daoquan Chen, Weicong Hong, Xiuze Zhou. "Transformer Network for Remaining Useful Life Prediction of Lithium-Ion Batteries", <i>IEEE Access</i> , 2022 Publication	<1 %

- 53 Huanwei Xu, Lingfeng Wu, Shizhe Xiong, Wei Li, Akhil Garg, Liang Gao. "An improved CNN-LSTM model-based state-of-health estimation approach for lithium-ion batteries", Energy, 2023 $<1\%$
-

Exclude quotes Off
Exclude bibliography Off

Exclude matches Off

ABSTRACT

This thesis presents a comprehensive exploration of data-driven methodologies for predicting the Remaining Useful Life (RUL) and State of Health (SOH) of lithium-ion batteries, crucial for the broader adoption of electric vehicles (EVs). Addressing the environmental concerns associated with fossil fuels, EVs emerge as a sustainable alternative, contingent upon advancements in battery technology to ensure safety, reliability, and cost-effectiveness. The study leverages the NASA and Oxford Battery Degradation datasets to develop and validate a predictive model that integrates advanced machine learning techniques, including Long Short-Term Memory (LSTM) networks and Transformers.

The research identifies the challenges in accurately predicting battery RUL and SOH, such as non-linear behavior, diverse operational conditions, and limited historical data. To overcome these, the thesis proposes a novel neural network model, AttentiveLSTM, which combines LSTM with Transformer architectures to enhance feature extraction and time-series prediction capabilities. This model is evaluated through rigorous testing on distinct datasets, demonstrating its effectiveness in capturing complex battery degradation patterns.

Key contributions include the development of a robust framework that simultaneously predicts SOH and RUL, and the introduction of an innovative objective function that integrates Denoising Autoencoder (DAE) loss with prediction loss. The results, indicated by lower Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Relative Error (RE), suggest significant improvements over existing methods. This work not only advances battery health prediction technologies but also supports the transition to sustainable transportation solutions by enhancing the reliability and efficiency of EV batteries.

Keywords: battery health prognosis, lithium-ion battery, electric vehicle, battery management systems, transformers, state of health, remaining useful life

Contents

Title	Page No.
CERTIFICATE	i
ACKNOWLEDGEMENT	ii
PLAGIARISM REPORT	iii
ABSTRACT	xii
LIST OF CONTENT	xiii
LIST OF FIGURES	xvi
LIST OF TABLES	xvii
1 Introduction and Background	1
1.1 Motivation	1
1.2 Key Challenges	1
1.3 Problem Addressed in Thesis	2
1.4 Approach to the Problem and Organization of Thesis	2
1.5 Background	2
1.5.1 Lithium-ion Batteries	2
1.5.2 Battery Management System	3
1.5.3 Battery related Terminology	4
2 Literature Review	6
2.1 Literature Survey	6
2.2 Review of SOH Estimation Methods	6
2.3 Review of RUL Estimation Methods	9
2.4 Review On the Basis Of Different Methods Used	10
2.4.1 Model Based Methods	10
2.4.2 Data Driven Methods	11
2.4.3 Experimental Methods	12
2.5 Gap Analysis And Contributions	13

3	Methodology	15
3.1	Literature Review Methodology	15
3.1.1	Research Questions	15
3.1.2	Review Protocol	16
3.2	Machine Learning Models	16
3.2.1	LSTM	16
3.2.2	GRU	17
3.2.3	Transformer	17
3.2.4	Random Forest	17
3.3	Proposed Method	17
3.4	Dataset Description	18
3.4.1	NASA Dataset	20
3.4.2	Oxford Battery Degradation Dataset	20
3.5	Dataset Preprocessing	21
3.5.1	NASA Dataset Preprocessing	21
3.5.2	Oxford Dataset Preprocessing	21
3.6	Experimental Setup	22
3.6.1	NASA Dataset	22
3.6.2	Oxford Battery Degradation Dataset	22
3.7	Exploratory Data Analysis	23
3.7.1	NASA Dataset EDA	23
3.7.2	Oxford Battery Degradation Dataset EDA	24
4	Results and Discussions	26
4.1	Evaluation Criteria	26
4.1.1	MAPE	26
4.1.2	RMSE	26
4.1.3	R squared	27
4.1.4	Relative Error	27
4.2	Hyperparameter Optimization Results	27
4.3	SOH Estimation Results	30
4.4	RUL Prediction Results	32
5	Conclusion and Future Work	34
5.1	Conclusion	34
5.2	Future Work	34
5.3	Task, Achievements and Possible Beneficiaries	35
	REFERENCES	36

BIO-DATA	40
Appendices	42
A Exploratory Data Analysis of NASA Dataset and Results from Other Algorithms	43
B Code Attachments	46

LIST OF FIGURES

1.1	Schematic Diagram of Battery Management System	4
2.1	Machine Learning Methods Used to Predict Battery Degradation based on Literature Review	9
2.2	Year-wise count of publications on SOH and RUL prediction.	10
3.1	Proposed Method for prediction of SOH and RUL	18
3.2	Exploratory Data Analysis on NASA Dataset	23
3.3	SOH vs Discharge Cycle of different groups for NASA Dataset	24
3.4	Exploratory Data Analysis on Oxford Battery Degradation Dataset	24
4.1	AttentiveLSTM-based SOH Prediction on NASA Dataset when trained on 50% of the dataset	28
4.2	AttentiveLSTM-based SOH Prediction on NASA Dataset when trained on 70% of the dataset	29
4.3	Performance of AttentiveLSTM and Baseline Models on Test Dataset of Oxford Battery Degradation	32
A.1	Linear Regression-based SOH Prediction on NASA Dataset when trained on 50% of the dataset	43
A.2	Linear Regression-based SOH Prediction on NASA Dataset when trained on 50% of the dataset	44
A.3	Linear Regression-based SOH Prediction on NASA Dataset when trained on 70% of the dataset	45

LIST OF TABLES

2.1	Summary of Battery Health Estimation Methods	7
3.1	Research Questions	16
3.2	Oxford Dataset Description	20
4.1	Comparison of RMSE between Linear Regression and LSTM models for SOH prediction on the NASA dataset using different training data proportions (50% and 70%)	30
4.2	MAE Evaluation: Linear Regression vs. LSTM for SOH Prediction on NASA Battery Data (50% vs. 70% Training Split)	30
4.3	AttentiveLSTM vs. Baseline Models for RUL Predictions	31
4.4	Comparison of RUL prediction performance between LSTM and At- tentiveLSTM (Transformer-based) models.	33

Chapter 1

Introduction and Background

1.1 Motivation

The ubiquitous reliance on fossil fuels has become a major environmental concern, with greenhouse gas emissions contributing significantly to climate change. In response, a global shift towards cleaner and renewable energy sources, such as solar, wind, and tidal power, is gaining momentum. Within the transportation sector, electric vehicles (EVs) powered by lithium ion batteries are emerging as a compelling alternative to gasoline and diesel-powered internal combustion engine vehicles (ICVs). Despite the clear environmental benefits of EVs, their widespread adoption faces challenges related to battery health, safety, and cost. Lithium-ion batteries, while offering high energy density and compact size, have a limited lifespan and a potential for catastrophic failure, raising significant concerns for EV users. To address these concerns and optimize battery management systems, accurate predictions of the Remaining Useful Life (RUL) and State of Health (SOH) of lithium-ion batteries become crucial.

1.2 Key Challenges

Predicting the RUL and SOH of EV batteries presents a myriad of challenges stemming from the complex interplay of various factors. One significant challenge lies in accurately modeling battery degradation, which is influenced by diverse factors such as temperature fluctuations, charging/discharging cycles, and usage patterns. Additionally, EV batteries exhibit nonlinear behavior and dynamic characteristics, making it difficult to develop precise predictive models. Limited availability of historical data and variability in operating conditions further complicate the prediction process. Moreover, the lack of standardized testing protocols and the need for real-time monitoring systems pose significant hurdles in accurately assessing RUL and SOH. Addressing these challenges necessitates interdisciplinary efforts involving advanced data analytics, materials science, and battery technology innovations to enhance the accuracy and reliability of predictive models for EV battery management.

1.3 Problem Addressed in Thesis

This thesis comprehensively examines data-driven methodologies employed for RUL and SOH prediction in lithium-ion batteries. On comprehensively reviewing the literature on the subject, a lack of an all-inclusive robust framework applicable for both SOH and RUL prediction is seen. The thesis aims to solve that by making a robust framework, with effective data cleaning and feature extraction modules, that predicts SOH and RUL simultaneously.

1.4 Approach to the Problem and Organization of Thesis

The review examines different SOH and RUL estimation methods and provides a comprehensive analysis of existing research, encompassing research trends, limitations, and future directions. By comprehensively exploring these aspects, this review aims to contribute to significant advancements in battery health prediction and management techniques. Ultimately, these advancements pave the way for the development of safer, more reliable, and efficient electric vehicles, fostering a greener and more sustainable future for transportation. Furthermore, this review aims to bridge the gap between the increasing demand for EVs and the concerns surrounding battery health. By providing a comprehensive analysis of existing research and highlighting areas for future research, this review aims to equip stakeholders with the necessary knowledge and insights to accelerate advancements in battery technology and facilitate the widespread adoption of EVs.

1.5 Background

1.5.1 Lithium-ion Batteries

The initial attempts at electric vehicles were hampered by the limitations of lead-acid batteries, which offered a meager energy density of 30 Wh/kg and a lifespan of only 300 charging cycles. This paled in comparison to gasoline, boasting a tenfold higher energy density (10,000 Wh/kg), leading to bulky, heavy EVs with limited range. To overcome these challenges, the industry transitioned to lithium-ion batteries, a game-changer for electric transportation. Lithium-ion batteries boast a significantly higher energy density, allowing for lighter, more compact designs and longer driving ranges in EVs. Additionally, they offer a vastly extended lifespan, lasting for thousands of cycles compared to lead-acid's meager 300. Furthermore, lithium-ion batteries have a lower environmental impact, eliminating the harmful lead emissions associated with their lead-acid counterparts. The text delves into the inner workings of lithium-ion batteries, explaining how they function through the movement of lithium ions between the positive and negative

electrodes during charging and discharging cycles. While the formation of a protective film on the negative electrode during the first charge can lead to a slight decrease in battery capacity over time, this minor drawback is far outweighed by the numerous advantages lithium-ion batteries offer. Overall, their superior energy density, extended lifespan, and environmental benefits solidify lithium-ion batteries as the clear choice for powering modern electric vehicles.

1.5.2 Battery Management System

Within the heart of an electric vehicle (EV) lies a crucial component, the battery management system (BMS). Often referred to as the silent guardian of the battery, the BMS plays a vital role in ensuring its safe and efficient operation. Operating much like the conductor of an orchestra, the BMS constantly monitors and regulates the battery's performance, akin to the conductor overseeing the various instruments. This silent guardian employs a dedicated sampling unit, which acts like an attentive audience member, gathering crucial information about the battery's well-being. This information includes the battery's temperature, voltage, and current, akin to listening to the pitch, volume, and rhythm of the instruments. By analyzing this data using sophisticated algorithms, the BMS acts as a skilled interpreter, estimating the battery's state of health, charge level, power output, and remaining lifespan. This understanding empowers the BMS to make informed decisions for the battery's well-being, just like a conductor adjusts the tempo and dynamics to optimize the overall performance of the orchestra. Furthermore, the BMS assumes several critical responsibilities beyond mere monitoring. It acts as a vigilant protector, constantly checking the battery's health and safety and taking necessary actions if it detects any abnormal conditions, akin to the conductor ensuring harmony and preventing any discord within the orchestra. The BMS also functions as a dedicated data analyst, meticulously acquiring and storing data about the battery's performance for future analysis and improvement. Additionally, it acts as a skilled energy manager, optimizing the battery's energy usage to maximize its efficiency and extend its lifespan, similar to the conductor ensuring a smooth and efficient flow of music. Finally, the BMS plays the role of a thermal conductor, meticulously maintaining the battery's temperature within a safe operating range, similar to the conductor ensuring the musicians perform in the optimal environment for optimal performance. In conclusion, the BMS, through its diverse and critical functionalities, ensures the safe, reliable, and efficient operation of the battery, ultimately contributing to a smooth and enjoyable driving experience in EVs. Fig 1.1 given below represents the schematic diagram of a typical battery management system.

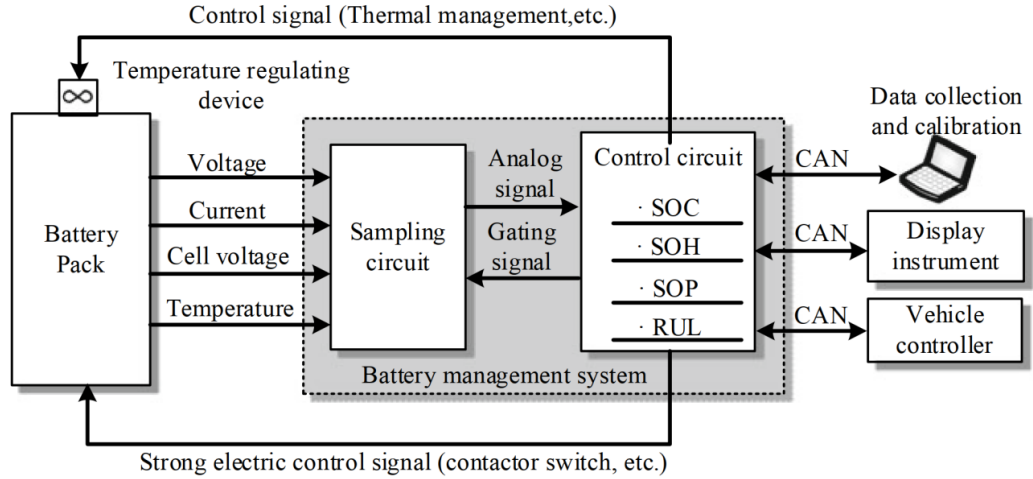


Figure 1.1: Schematic Diagram of Battery Management System

1.5.3 Battery related Terminology

Here are some key terminologies related to battery management systems (BMS) and lithium-ion batteries:

- **State of Health (SOH):** SOH represents the overall health or condition of a battery, indicating the ratio of its present maximum charge capacity to the maximum charge capacity at the beginning of its life.

$$SOH = \frac{Q_{act}}{QR} \times 100\%$$

- **State of Charge (SOC):** SOC refers to the available capacity or charge in the battery relative to its total capacity. It is typically expressed as a percentage.

$$SOC(t) = SOC_{init} \pm 1 - \frac{\int_0^t \eta_i(t) dt}{Crated}$$

- **Depth of Discharge (DOD):** DOD represents the percentage of the battery's capacity that has been discharged during a load cycle. It is expressed as a percentage of the maximum capacity.
- **State of Power (SOP):** SOP indicates the ratio of peak power to nominal power of the battery. Peak power refers to the maximum power the battery can deliver persistently within specified constraints.

$$SOP = \frac{Peak\ power}{Nominal\ power}$$

- **State of Temperature (SOT):** SOT reflects the thermal condition of the battery, considering factors such as heat generation, transfer, and dissipation. It is crucial for managing thermal runaway and ensuring safe operation.

$$SOT = f(heat\ flow, distribution)$$

- **State of Energy (SOE):** SOE determines the remaining energy in the battery, allowing prediction of the remaining driving mileage in EV applications.

$$SOE(t) = SOE_{init} + 1 + \frac{\int_0^t P(\tau) d\tau}{E_{rated}}$$

- **Remaining Useful Life (RUL):** RUL represents the remaining time or number of load cycles before the battery reaches the end of its operational life. It is essential for predictive maintenance and ensuring continuous operation.

$$RUL = TEOL - T_{present}$$

- **Beginning of Life (BOL):** BOL signifies the initial operational state of a battery when it has just started operating.
- **End of Life (EOL):** EOL indicates the point at which a battery no longer meets its operational specifications.
- **State of Function (SOF):** SOF denotes the battery's peak power capability and its ability to operate within safe limits.
- **Remaining Driving Range (RDR):** RDR is the distance an electric vehicle can travel before the battery reaches its end of life.

Chapter 2

Literature Review

2.1 Literature Survey

Table 2.1 presents a summary of various approaches employed by different researchers for battery health estimation. The table includes authors' names, publication years, battery types, parameter conditions, models/methods utilized, and the average prediction error.

2.2 Review of SOH Estimation Methods

Various documents classify techniques for State of Health (SOH) estimation differently. Some categorize them as experiment-based, model-based, or machine-learning-based methods, while others classify them into direct, indirect, adaptive, and data-driven methods. Furthermore, SOH estimation techniques are sometimes categorized as physically based, empirically based, or using Incremental Capacity Analysis (ICA) and Differential Voltage Analysis (DVA), along with data-driven approaches. Model-based approaches, when combined with other algorithms, exhibit high accuracy but are sensitive to minor system changes and unsuitable for real-time dynamic conditions, heavily relying on model parameters for accuracy. Experimental methods encompass both direct and indirect techniques. Direct methods demand precise test equipment and controlled environments for accurate results, typically measuring internal resistance or charging capacity offline. In contrast, indirect methods involve generating ICA-DVA incremental curves during experiments and performing peak analysis. However, these methods are not suitable for real-time operation and may induce temperature fluctuations, necessitating constant current charging.

Rest periods during long cycling cycles contribute to cell capacity recovery, with varying recovery rates based on State of Charge (SOC) levels. For instance, a 0% SOC exhibits better recovery than 10% or 20% SOC levels. This phenomenon, known as self-recovery or regeneration, can be detected and leveraged to improve long-term predictions using techniques like particle filtering. Prognostics for battery health rely on past, present, and anticipated usage data, incorporating historical operational data,

Table 2.1: Summary of Battery Health Estimation Methods

Paper	Year	Battery Type	Parameter	Model/ Method	Description	Average Error
Hung, Min-Hsuan, et al.	2014	2150 mAh Samsung Li-ion (NCA)	Change in voltage/current	Dynamic Impedance	Calculating SOC/SOH	not specified
He, Z.; Gao, M.; Ma, G.; Liu, Y.; Chen, S.	2014	3.7 V, 6000 mAh Li-Mn	Terminal voltage	Dynamic Bayesian Network	Capacity test, lifecycle	5%
Xiong, R.; Sun, F.; Chen, Z.; He, H.	2014	32 Ah Li-ion	Change in RC parameter	Lumped model, EKF	Capacity estimation	2%
Hu, X.; Li, S.E.; Jia, Z.; Egardt, B.	2014	8 Li-ion (NMC)	Discharge curve voltage	Sample entropy	Capacity prediction	2%
Klass, V.; Behm, M.; Lindbergh, G.	2014	Li-ion (PHEV)	Current, temp., SOC	SVM	Dynamic conditions	not specified
Guo, Z.; Qiu, X.; Hou, G.; Liaw, B.Y.; Zhang, C.	2014	Li-ion (LMO)	Charging rate, voltage, temp.	ECM	Model identification	2%
Han, X.; Ouyang, M.; Lu, L.; Li, J.	2014	Various Li-ion (NMC/LTO, LFP/C, LMO/C)	Temp. variation	Genetic Algorithm	SOH estimation	1%

Zou, Y.; Hu, X.; Ma, H.; Li, S.E.	2015	Li-ion (NMC)	RC ECM, EKF	HPPC, FUDS, DST	Polarization resistance	not speci- fied
You, G.-W.; Park, S.; Oh, D.	2016	Battery (EV)	Driving pro- files	NN	Classification/ regression	2.18%
Keil, P.; Jossen, A.	2017	2.8 Ah Pana- sonic Li-ion (NCA)	Driving load profiles	Real-time profile	Aging effects	not speci- fied
Zhang, X.; Wang, Y.; Liu, C.; Chen, Z.	2018	38 Ah Li-ion (NMC)	Electro- chemical model	PSO-GA	Noise reduction	not speci- fied
Yang, J.; Xia, B.; Huang, W.; Fu, Y.; Mi, C.	2018	3.2 V, 2.5 Ah Li-ion (LFP)	Voltage/ cur- rent at CV mode	RC ECM	Capacity correla- tion	2.5%
Duan et. al	2023	38 Ah Li-ion (NMC)	Charging voltage slope	VFOS- ELM	SOH estimation	not speci- fied
Zhao et. al	2022	2 Ah Li- ion	Terminal voltage derivative	LSTM, GPR	Health character- istics	not speci- fied
Cheng et. al	2021	1100 mAh Li-ion	Voltage, cur- rent, temp.	EMD, B- LSTM	Parameter esti- mation	not speci- fied
Jafari et. al	2023	1300 mAh Li-ion	Voltage, cur- rent, temp.	Harris Hawks Optimiza- tion	RUL prediction	not speci- fied
Marri et.al	2023	6800 mAh Li-ion	Voltage/ cur- rent limits	Regression, SVR, RFR	Feature selection	not speci- fied

sensor feedback, and environmental conditions.

State transition models aid in noise elimination, while short-term current pulse tests offer insights into SOH estimation. Machine learning techniques, such as Bayesian Ridge Regression (BRR), Gaussian Process Regression (GPR), Random Forest (RF), and Deep Neural Networks (DNN), provide diverse capabilities for SOH estimation, with different strengths and weaknesses. Recovery voltage, influenced by temperature and aging, emerges as a new parameter for SOH estimation, while feature selection is crucial to prevent performance degradation.

Hybrid methods, combining multiple algorithms, enhance efficiency and accuracy while reducing Battery Management System (BMS) costs. These approaches leverage filter techniques to mitigate dataset noise and employ ANN or LSTM for complex or time-series data, respectively. State estimation benefits from techniques like Kalman Filtering (KF) and Adaptive Unscented Kalman Filtering (AUKF), while cell imbalances and various degradation factors impact battery state and longevity. Overcharging and over-discharging lithium-ion batteries can lead to severe consequences, including reduced lifespan and safety hazards.

2.3 Review of RUL Estimation Methods

Several machine learning models offer non-probabilistic RUL prediction, providing a single point estimate. ARIMA models based on past observations may struggle with the non-linear degradation of batteries. Artificial Neural Networks (ANNs), inspired by the human brain, come in various architectures like LSTMs and DNNs, showing promise in RUL prediction. Support Vector Machines (SVMs), typically used for classification, can be adapted for regression using kernel functions for RUL prediction. Fig. 2.1 denotes the number of papers in which the authors have used different Machine learning methods to predict the battery degradation cycle.

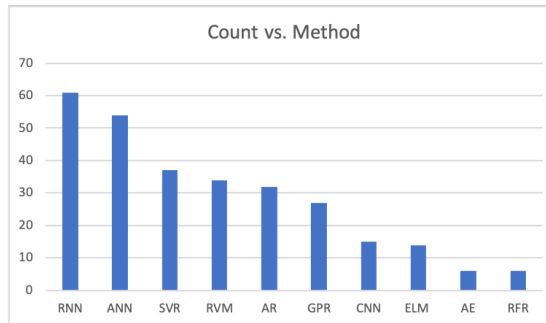


Figure 2.1: Machine Learning Methods Used to Predict Battery Degradation based on Literature Review

These models leverage the degradation profiles of similar batteries to predict RUL for a new battery. Dynamic Time Warping (DTW) technique aligns similar degradation

patterns, even if they occur at different time scales, and has been combined with neural networks for RUL prediction. Similarity-based models (SBMs) have also been explored for RUL prediction in different contexts.

In contrast, probabilistic models offer both a point estimate and a confidence interval for RUL prediction, providing additional information about the uncertainty associated with the prediction. Gaussian Process Regression (GPR) and Relevance Vector Machines (RVMs) are such methods, operating within a Bayesian framework. These probabilistic models offer valuable insights into the certainty of RUL predictions, aiding decision-making processes. Researchers have also explored combining different machine learning techniques, such as Support Vector Regression (SVR) with a Particle Filter (PF), Neural Networks with Particle Filters, and combinations of Relevance Vector Machine (RVM), Particle Filter, and AR models, to potentially improve RUL prediction accuracy. Figure 2.2 given below demonstrates the numbers of papers published for prediction of both SOH and RUL.

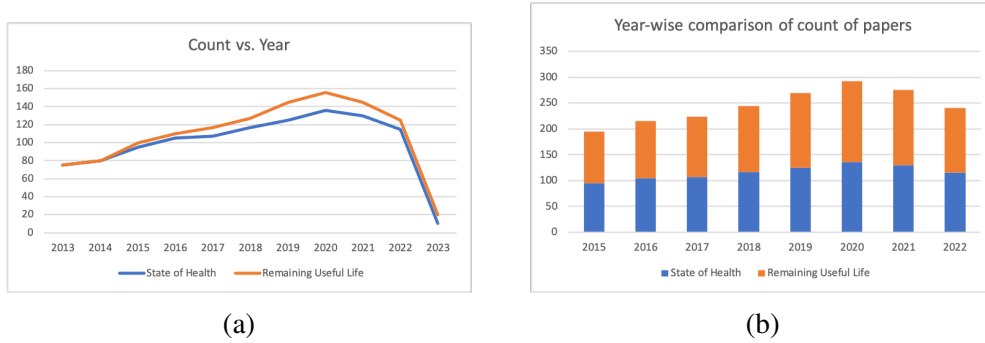


Figure 2.2: Year-wise count of publications on SOH and RUL prediction.

2.4 Review On the Basis Of Different Methods Used

2.4.1 Model Based Methods

Gao et al [1] explored techniques to estimate the state of lithium-ion batteries used in electric vehicles, covering battery chemistries, management systems, data-driven and model-based estimation methods for state of charge and health, research trends, open datasets, and key challenges.

Xiong et al [2] proposed an advanced time scale separation algorithm that improves battery modeling, implements multi-scale extended Kalman filtering for parameter and state-of-charge estimation on different timescales, and enables real-time accurate capacity and state-of-charge estimation, validated through lithium-ion polymer battery experiments.

Zou et al [3] proposed a combined state of charge (SOC) and state of health (SOH) estimation method for lithium-ion batteries, utilizing a first-order RC model, extended

Kalman filters on different timescales for real-time SOC and offline SOH estimation, demonstrating results over the battery's lifetime using extensive testing data.

Guo et al [4] proposed an adaptive charging curve transformation method to quantify lithium-ion battery capacity fade and derive a time-based parameter for accurate, chemistry-agnostic state-of-health estimation, achieving less than 3% error across different cell types, temperatures, and discharge rates.

Han et al [5] conducted cycle life experiments on five commercial lithium-ion cells under EV operating conditions (45°C and 5°C), build cycle life models using genetic algorithms to fit capacity loss trends, and proposed a state-of-health estimation method combining online capacity modeling and regular calibration for automotive applications.

Zhang et al [6] proposed a method for accurate state-of-health estimation of lithium-ion battery packs in electric vehicles, utilizing particle swarm optimization-genetic algorithms for model parameterization, particle filtering for state-of-charge and open-circuit voltage estimation, and recursive least squares for cell capacity updates, validated through experimental driving cycles.

Yang et al [7] proposed a method for accurate state-of-health estimation of lithium-ion battery packs in electric vehicles, utilizing particle swarm optimization-genetic algorithms for model parameterization, particle filtering for state-of-charge and open-circuit voltage estimation, and recursive least squares for cell capacity updates, validated through experimental driving cycles.

Marri et al [8] compared classical machine learning strategies like multiple linear regression, polynomial regression, support vector regression, and random forest for lithium-ion battery state-of-health estimation, with support vector regression performing best, and also discusses feature selection based on partial charging time between voltage intervals.

2.4.2 Data Driven Methods

Xiaosong et al [9] presented an improved sample entropy-based capacity estimator for lithium-ion battery prognostics and health management in electric vehicles, demonstrating an average 2% relative error using aging datasets across multiple cells.

Klass et al [10] developed and experimentally validated a data-driven method using a support vector machine battery model for online estimation of lithium-ion battery capacity and resistance as state-of-health indicators, suitable for on-board implementation in electric vehicles under typical operating conditions.

Park et al [11] proposed a data-driven approach to estimate lithium-ion battery state-of-health on the fly for electric vehicles using current, voltage, temperature data and historical distributions, achieving highly accurate results with less than 2.18% average

error under real-world driving conditions.

Zhao et al [12] proposed a method using LSTM neural networks and Gaussian process regression to jointly predict lithium-ion battery state of health and remaining useful life, achieving high accuracy and reliability.

Cheng et al [13] proposed a combined empirical mode decomposition (EMD) and backpropagation long-short-term memory (B-LSTM) neural network approach for accurate estimation of battery state of health (SOH) and remaining useful life (RUL) prediction, demonstrating its high robustness, accuracy, and applicability.

Roman et al [14] designed and evaluated a machine learning pipeline using parametric and non-parametric algorithms to reliably estimate lithium-ion battery state of health (capacity fade) with confidence intervals. The best model achieved 0.45% RMSE when applied to fast-charged cells, showcasing a scalable data-driven approach for real-time battery diagnostics.

Zhang et al [15] proposed a novel algorithm using an extended Kalman particle filter with a transformed double exponential degradation model and an autoregressive model to accurately predict the remaining useful life of lithium-ion batteries, outperforming other existing algorithms in prediction accuracy.

Qiu et al [16] proposed using backward smoothing square root cubature Kalman filter and multiscale hybrid Kalman filter for accurate state of charge and health estimation, combined with an improved cuckoo search particle filter algorithm to improve remaining useful life prediction accuracy and confidence for lithium-ion batteries.

Wang et al [17] developed a Lebesgue-sampling-based extended Kalman filter (LS-EKF) algorithm for computationally efficient state-of-health (SOH) and state-of-charge (SOC) estimation and remaining useful life prediction of lithium-ion batteries in battery management systems, demonstrating good accuracy and reduced computational cost.

Deng et al [18] proposed a novel method for accurate lithium-ion battery state-of-health estimation by theoretically selecting and mathematically screening relevant features, employing a new training set design for Least Squares Support Vector Machines to handle multiple operating conditions, validated using NASA datasets.

2.4.3 Experimental Methods

Hung et al [19] proposed a dynamic impedance method for real-time state of charge (SOC) estimation and a projection method for state of health (SOH) estimation of lithium-ion batteries, integrating these techniques into a battery management system with experimental validation.

Keil et al [20] conducted an experimental aging study investigating the impact of regenerative braking, temperature, state-of-charge, and driving load dynamics on lithium-

ion battery degradation in electric vehicles, revealing insights on capacity fade, resistance increase, and strategies to minimize battery aging under real-world EV conditions.

Severson et al [21] generated a dataset of lithium-ion battery cells with varying cycle lives under fast-charging. Using early discharge voltage curves, they applied machine learning models to accurately predict and classify the cells' cycle life, showcasing the potential of combining data generation and data-driven modeling for complex systems.

Tian et al [22] reviewed aging causes and state of health (SOH) prediction methods for lithium-ion batteries in electric vehicles. They discuss the benefits and limitations of classification-based SOH prediction approaches and provide suggestions for implementing these methods in actual industrial production environments.

2.5 Gap Analysis And Contributions

A strong framework that addresses the problems of State of Health (SOH) assessment as well as Remaining Useful Life (RUL) prediction should comprise a module for data purification, another for feature extraction and selection, and a core algorithm module that can forecast RUL in addition to evaluating SOH.

In terms of the overall framework, traditional methods are frequently incomplete because they leave out important modules, which restricts their ability to address the State of Health (SOH) or Remaining Useful Life (RUL) issues independently. Changes to the model parameters and framework structure are necessary to address both tasks at the same time.

The framework that Zhao et al. [12] discuss in their literature is limited to solving the RUL prediction problem. This literature forecasts the capacity for the next time step by using capacity data from a certain time frame as model input. Nevertheless, this system does not have the capacity to extract feature data from raw inputs like current, voltage, and temperature, which is a critical component for State of Health (SOH) assessment. This is because it ignores a critical data processing module. Moreover, the fundamental technique used in this framework, LSTM+BLS, is extremely basic, which leads to poor feature extraction performance and inaccurate SOH information capture. Significant modifications to the parameters and structure of the basic model inside the framework are necessary to enable Remaining Useful Life (RUL) prediction.

Practically speaking, the framework's feature extraction/selection and data cleaning modules' efficacy is rather constrained.

State of Health (SOH) estimation has been done using traditional machine learning techniques like SVM and GPR. These studies, however, don't deal with the difficulties involved with online internal resistance and battery capacity measuring. In order to facilitate the online collection of discharge process data for battery SOH estimate, Hui et

al. [23] presented novel features. However, they failed to account for the difficulties and unpredictability involved in gathering discharge data for both SOH and Remaining Useful Life (RUL) estimates. In order to address the problems mentioned in the reference, Zhou et al. [24] developed the mean voltage falloff (MVF) feature; however, they neglected to take into account how noisy data could affect assessment predictions. In order to reduce data noise, Cheng et al. [25] used the Empirical Mode Decomposition (EMD) algorithm and combined it with an LSTM algorithm for RUL prediction. Nevertheless, multi-mode aliasing can affect EMD. By combining deep learning techniques like Convolutional Neural Network (CNN), LSTM, and self-attention with directly measured data, such as voltage and current, researchers like Chen et al. [26] were able to provide results with a high degree of precision. While this method yields precise outcomes, it has high processing requirements, which makes it difficult to update online implementations. These problems render these approaches less useful.

The lack of an optimal multi-task learning core algorithm designed specifically for State of Health (SOH) assessment and Remaining Useful Life (RUL) prediction is one of the primary obstacles.

The SOH assessment process requires a complex mapping between capacity labels and the health parameters of lithium batteries, necessitating the model to have strong capabilities in feature information extraction and mapping. On the other hand, long-term, nonlinear time series forecasting presents a problem for RUL prediction. Time series learning must be done exceptionally well because the model must predict the subsequent capacity value using information from earlier time periods. Both SOH assessment and RUL prediction must be seamlessly integrated by an ideal multi-task learning system.

Chapter 3

Methodology

3.1 Literature Review Methodology

Systematic Literature Review strategy proposed by Kitchenham et. al [27] is adopted for this literature review.

3.1.1 Research Questions

The process starts by establishing Research Questions. To determine the correlation between ML and EV battery four Research Questions are proposed for this study.

1) RQ1: To pinpoint the most influential researchers, universities, and working groups in SOH, SOC and RUL prediction. This question aims to map their timeline and geographic locations to better understand their research motivations and concerns.

2) RQ2: To correlate different algorithms that have been tested in the field of SOH and RUL prediction to address our specific problem. This step seeks to identify new and robust algorithms to enhance forecasting accuracy and comprehend the underlying principles of these strategies.

3) RQ3: To describe the datasets used in the experiments, including their volume and integrity. This question helps in understanding the various data sources, including electricity, weather, astronomical, and other forms of data, and their relevance to algorithm performance.

4) RQ4: To describe the development of new technologies worldwide and their interconnections in the context of prediction of Battery Failure. This question provides insights into potential research barriers that may emerge in the field.

Table 3.1: Research Questions

RQ1	To analyze the research landscape of SOH, SOC, and RUL estimation using ML techniques.
RQ2	To identify the most influential researchers, institutions, and collaborations in the field of ML-based SOH, SOC, and RUL estimation.
RQ3	To evaluate the effectiveness of different ML algorithms for SOH, SOC, and RUL estimation.
RQ4	To explore the challenges and opportunities associated with applying ML for SOH, SOC, and RUL estimation in BMS.

3.1.2 Review Protocol

The review protocol outlines clearly defined steps to determine answers to the established research questions.

1. Determining research questions at the beginning of the protocol, as described.
2. Derivation of relevant search strings capable of answering the previously established questions. These strings are then inserted in the Web of Science database, considered the universal set of all existing databases to attain maximum results.
3. Definition of inclusion and exclusion criteria to filter the papers out.
4. Use of individual filter strings to remove duplicates and outdated results while providing focused and most relevant scientific results, among others.
5. Further filtration of papers based on their title, abstracts, and conclusions. Only those papers were considered further which focused on Battery failure prediction by developing, using, or comparing state-of-the-art machine learning algorithms. This step determines which papers are to be discarded and which are shortlisted for full reading.
6. Full reading of all the remaining papers.
7. Building of a database to store the results.
8. Key papers addressing Battery failure prediction using machine learning algorithms were shortlisted for further analysis.

3.2 Machine Learning Models

3.2.1 LSTM

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) architecture that is well-suited for handling sequence data, such as time series or natural language text. Unlike traditional RNNs, LSTM networks have the ability to learn

long-term dependencies in data, making them particularly effective for tasks involving context or memory over time. The key feature of LSTM networks is their gated mechanism, which allows them to selectively learn and retain information over multiple time steps. This mechanism consists of three gates: the input gate, forget gate, and output gate, each responsible for regulating the flow of information through the network.

3.2.2 GRU

The Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) architecture designed to address the limitations of traditional RNNs, such as vanishing gradients and difficulty in capturing long-term dependencies. GRUs, like LSTMs, utilize gated mechanisms to control the flow of information through the network. However, GRUs have a simpler architecture compared to LSTMs, with only two gates: the update gate and the reset gate.

3.2.3 Transformer

The Transformer, a revolutionary neural network architecture, redefined sequence modeling by solely relying on attention mechanisms, dispensing with recurrent connections. At its core, attention mechanisms allow each element in the sequence to selectively attend to relevant information, facilitating efficient long-range dependency capture. Key features include self-attention layers enabling global interactions, multi-head attention for diverse information aggregation, and positional encodings to preserve sequence order.

3.2.4 Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees using random subsets of data and features. By aggregating predictions from these trees, it delivers robust and accurate results, suitable for classification and regression tasks across diverse domains.

3.3 Proposed Method

Both SOH assessment and RUL prediction must be seamlessly integrated by an ideal multi-task learning system. The majority of current techniques include connecting an LSTM or GRU layer to CNN first. In fact, the algorithm may be strengthened to handle the intricacies of both SOH assessment and RUL prediction by incorporating both CNN for reliable feature extraction and LSTM for efficient time series processing. The Transformer approach was employed by Chen et al. [26] to obtain RUL; however, this method's feature extraction and mapping capabilities are insufficient to meet the

performance requirements of SOH estimation. Bidirectional long short term memory (Bi-LSTM) neural networks and the self-attention algorithm were coupled by Wang et al. [28] and Zhao et al. [29] using the board learning system (BLS) and LSTM. Despite having RUL prediction and SOH estimation functionality, research shows that these algorithmic models frequently have poor generalisation skills and significant computing needs. Models built with data from one kind of battery, for example, might not be able to forecast the SOH and RUL of another type of battery with enough accuracy. This shortcoming may be the result of these algorithms' inability to extract enough features, which suggests that they are unable to fully capture the subtler aspects of battery ageing. This study creates a novel technique called AttentiveLSTM by combining LSTM with Transformer to address the aforementioned difficulties.

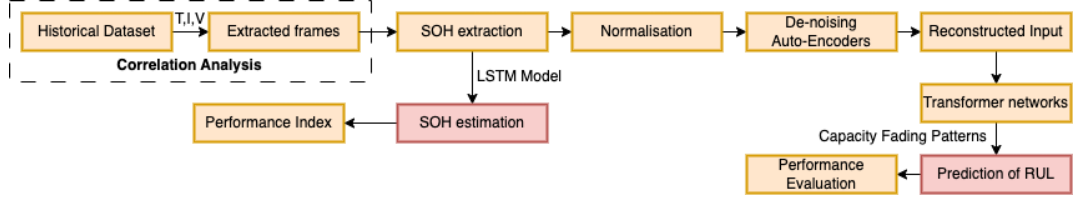


Figure 3.1: Proposed Method for prediction of SOH and RUL

After dataset preprocessing and feature engineering, relevant features, including the State of Health (SOH) are extracted. This SOH data is then normalized and fed into a Denoising Autoencoder (DAE) to remove noise. Using historical SOH data, we predict the SOH using LSTM methods. Transformer networks then analyze the reconstructed SOH data to learn features relevant to capacity fading. An objective function combines the DAE's ability to reconstruct data and the model's prediction accuracy. Finally, machine learning models use the learned features to predict the equipment's SOH and RUL, with the overall performance evaluated by a performance index. The entire flow diagram is illustrated in Fig 3.1.

Considering the research gaps and defined problems, our adopted code methodology enabled us to achieve our objectives. The algorithm outlines the sequential steps involved in data preprocessing, model training, hyperparameter tuning, and evaluation of model's accuracy.

3.4 Dataset Description

In our study, we embarked on an exploration and evaluation of two distinct datasets, leveraging the NASA and Oxford Battery Degradation datasets. Our primary objec-

Algorithm 1 Attentive LSTM Algorithm

Input Dataset D containing battery degradation information

Output Predicted State of Health (SOH) and RUL curves between $\hat{y}$ and y along with RMSE, MAE, RE

```
1: procedure ATTENTIVELSTM
2:   Load dataset  $D$ 
3:   Preprocess  $D$ 
4:   Initialize learning rate  $lr$ , number of layers  $num\_layers$ , hidden dimension
       $hidden\_dim$ , regularization  $\alpha_{reg}$ 
5:    $optimal\_hyperparameters$ 
6:    $best\_performance\_metric \leftarrow \infty$ 
7:   Split  $D$  into  $D_{train}$  and  $D_{test}$ 
8:   for  $e \leftarrow 1$  to  $N_{epochs}$  do
9:     for each batch  $b$  in  $D_{train}$  do
10:       $\hat{y} \leftarrow LSTM(\alpha_{reg}, \theta, lstm)$ 
11:       $loss \leftarrow MSE(\hat{y}, y)$ 
12:   for  $lr$  in  $lr\_values$  do
13:     for  $n$  in  $num\_layers\_values$  do
14:       for  $h$  in  $hidden\_dim\_values$  do
15:         Train transformer( $lr, n, h, \alpha_{reg}$ ) on  $D_{train}$ 
16:          $\hat{y} \leftarrow \text{transformer}(D_{test})$ 
17:          $loss \leftarrow MSE(\hat{y}, y)$ 
18:         if  $performance\_metric > best\_performance\_metric$  then
19:            $best\_performance\_metric \leftarrow performance\_metric$ 
20:            $optimal\_hyperparameters \leftarrow (lr, n, h, \alpha_{reg})$ 
21:   Plot  $y$  and  $\hat{y}$ 
22:   Print MAE, RMSE, and RE
```

tive was to develop and validate a predictive model for battery aging. By training our model on the NASA dataset and subsequently validating its performance on the Oxford dataset, we aimed to demonstrate its versatility and data-driven nature.

3.4.1 NASA Dataset

Machine learning models that estimate the State of Charge (SOC), State of Health (SOH), and Remaining Useful Life (RUL) of lithium-ion batteries (LIBs) frequently use data from NASA Ames Prognostics Centre of Excellence (PCoE). It includes four LIBs with the labels B0005, B0006, B0007, B0018, B0033, B0034, B0046, and B0047. [30] The data is run-to-failure. The charging process for these batteries involved maintaining a constant voltage of 1.5A until the voltage reached 4.2V, and then lowering the current to 20mA while maintaining a constant voltage. After that, discharge took place at a steady 2 amp current level until the voltage hit certain thresholds for every battery. Impedance values were determined using Electrochemical Impedance Spectroscopy (EIS) over a frequency range of 0.1 Hz to 5 kHz. The experiment recorded temperature, voltage, and current for every cycle and stopped when the battery’s capacity dropped to 70% of its starting value, signifying end-of-life (EOL).

3.4.2 Oxford Battery Degradation Dataset

The Oxford Battery Degradation dataset presents findings from aging assessments conducted on eight small lithium-ion pouch cells. Table 3.2 depicts the layer wise description of the Oxford Battery Degradation Set. Throughout the experimentation, all cells were consistently tested under a controlled temperature of 40°C, meticulously regulated by a thermal chamber. Testing protocols involved subjecting the cells to a continuous current constant voltage (CCCV) charging regimen, succeeded by discharge cycles adhering to an urban Artemis profile. Crucially, data collection occurred at regular intervals of 100 cycles. Notably, the observed increase in battery temperature across successive charge-discharge cycles serves as a pivotal indicator of battery aging, aligning with previous research findings.

Table 3.2: Oxford Dataset Description

Layer 1	Cells(1-8)
Layer 2	Cycle number of characterisation cycles
Layer 3	1-C charge (C1ch), 1-C discharge (C1dc), pseudo-OCV charge (OCVch), pseudo-OCV discharge (OCVdc)
Layer 4	time (t), voltage (v), charge (q), temperature (T)

3.5 Dataset Preprocessing

3.5.1 NASA Dataset Preprocessing

The NASA Dataset, stored in .mat files, is loaded into the Python environment using the `loadMat` function. This function reads the MATLAB file, extracts relevant columns, and organizes them into a structured format. Each entry in the structured data includes information such as battery type, temperature, time, and measurement data. For preprocessing of NASA Dataset, the following steps were taken:

1. **Time Extraction, Conversion, and Capacity Degradation:**

The `convert_to_time` function is employed to convert time data from strings into Python datetime objects. This conversion facilitates better manipulation and analysis of the time-related data. Subsequently, a function is utilized to filter out discharge cycles from the dataset and extract the corresponding capacity measurements. These capacity measurements are essential for analyzing battery degradation over time.

2. **Data Structuring and Sequence Building:**

After loading the data, it is structured into sequences that can be fed into the machine learning model. This is done using the `build_sequences` function, which creates overlapping windows of data points (sequences) that are used to predict the next data point in the sequence. For each battery dataset, sequences of a specified window size are created. This involves taking `window_size` consecutive capacity measurements as input features and the next capacity measurement as the target output.

3.5.2 Oxford Dataset Preprocessing

For offline training, the Oxford Battery Degradation Dataset 1 provides information on battery aging from eight tiny lithium-ion pouch cells. Testing took place at a constant temperature of 40°C inside a thermal chamber. The cells were charged using a constant voltage and current profile, and then they were discharged in cycles that mirrored the urban Artemis profile. The following steps were taken:

1. **Time Series Analysis for SOH Prediction:**

Since the data is collected chronologically for each cycle, time series analysis can be implemented to predict the State of Health (SOH) as the output. Some cycle data may be missing from the dataset during data collection, necessitating the use of time interpolation techniques to estimate these missing values by prioritizing nearby points over distant ones.

2. State of Health Definition and Normalization:

SOH is defined as the ratio of the current usable capacity of a battery to its nominal capacity. In this dataset, SOH data during the charge process $1 - C$ charge ($C1ch$) are utilized for training. Normalization is applied to all SOH data to enhance training efficiency and accuracy. `MinMaxScaler` is chosen as it maintains the original distribution shape without altering embedded information, preserving the scale of data relative to one another.

3. Sequence Grouping and Windowing:

The SOH data are then grouped into sequences as inputs to the model, with the sequence length representing the number of historical SOH data points used for prediction. To address missing cycles in the dataset, interpolation is used to fill in these gaps, leveraging previous trends and values. Additionally, windowing is employed as a preprocessing step to facilitate comparison between predicted and actual values during model training. This technique involves breaking the data into blocks of user-defined sizes, with the algorithm utilizing ten previous data points to predict subsequent ones.

3.6 Experimental Setup

3.6.1 NASA Dataset

The predictive model, defined in the `Net` class, includes an input layer, several hidden layers, and an output layer. The architecture is specifically designed to capture temporal patterns effectively in time-series data, crucial for predicting battery capacity degradation. The network initiates with a linear layer that maps the input features to a higher dimensional space. This is followed by multiple sequential layers, each comprising a linear layer paired with a ReLU activation function to introduce non-linearity, enhancing the model's ability to learn complex patterns. The final layer is a linear layer that outputs the predicted value. The model utilizes the Adam optimizer for weight updates, with a learning rate and weight decay configured to optimize convergence and prevent overfitting. The loss function used is Mean Squared Error (MSE), which quantifies the difference between the predicted capacities and the actual capacities. [30]

3.6.2 Oxford Battery Degradation Dataset

The machine learning models were constructed using PyTorch 2.0.0, a Python library tailored for deep learning tasks. Among the eight cells in the dataset, cells 1 through 4 were designated for training, yielding 255 datasets. The model was configured to retrospectively examine the preceding 10 data points. Consequently, the input structure took

the form of a 3D array with dimensions (255, 10, 1). Cells 5 through 8 were earmarked for testing the model's performance based on the Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

3.7 Exploratory Data Analysis

Exploratory Data Analysis (EDA) involves a detailed examination of the NASA and Oxford datasets, which contain time-series data on lithium-ion battery capacity degradation. Fig 3.2 (a) denotes the capacity degradation at 24°C for 4 cells, Fig 3.2 (b) represents the SOC vs Time Curve for a cell during charging and discharging and Fig 3.2 (c) represents the SOH evolution for the 8 cells. The objective of this analysis is to understand how battery capacity declines over time and under different charging conditions, which is crucial for predicting battery lifespan and optimizing usage strategies.

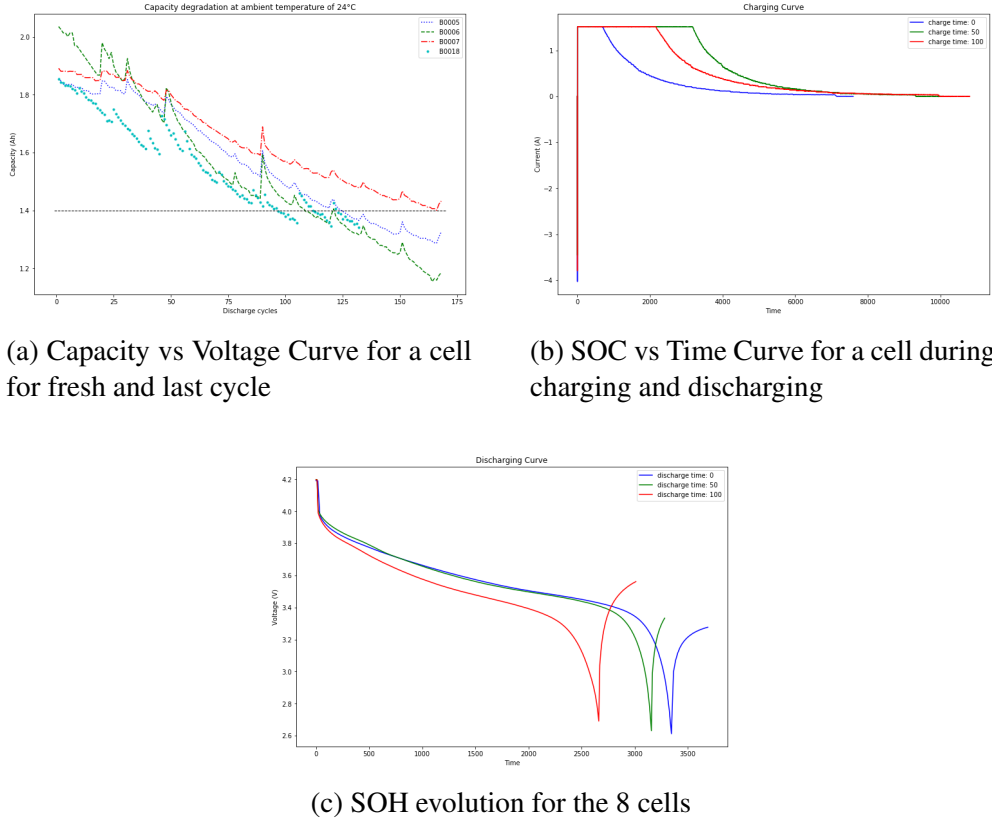
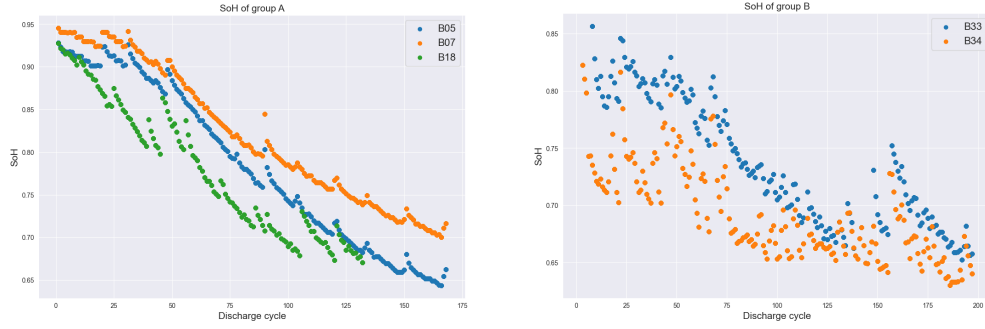


Figure 3.2: Exploratory Data Analysis on NASA Dataset

3.7.1 NASA Dataset EDA

The NASA dataset provides a valuable resource for analyzing the performance of lithium-ion batteries at a constant ambient temperature of 24°C. By examining the capacity degradation patterns across different battery cells, we can identify trends and anomalies in battery behavior. This analysis not only helps in assessing the reliability and

efficiency of battery cells but also aids in developing models to predict future performance based on historical data. [30] The graphs in the Fig 3.3 illustrate the capacity degradation over numerous discharge cycles, highlighting the variability in degradation rates among different battery cells. Fig 3.3 (a) represents the SOH vs Discharge Cycle for Cell B0006, Cell B0007 and Cell B0018 and Fig 3.3 (b) represents the SOH vs Discharge Cycle for Cell B0033 and B0044.

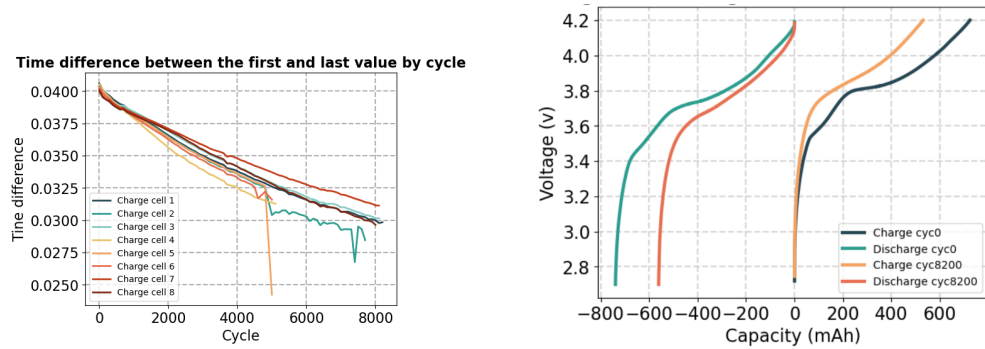


(a) SOH vs Discharge Cycle for Cell B0006, Cell B0007 and Cell B0018

(b) SOH vs Discharge Cycle for Cell B0033 and B0044

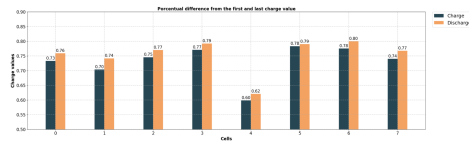
Figure 3.3: SOH vs Discharge Cycle of different groups for NASA Dataset

3.7.2 Oxford Battery Degradation Dataset EDA



(a) Time difference between first and last value by cycle when charging.

(b) Capacity vs Voltage Curve for a cell for fresh and last cycle



(c) Percentual Difference between first and last charge values for both the charge and discharge process.

Figure 3.4: Exploratory Data Analysis on Oxford Battery Degradation Dataset

The Oxford dataset offers a comprehensive view of the performance characteristics of lithium-ion batteries across their lifecycle. By analyzing changes in voltage, capacity, and state of charge (SOC) over numerous charging and discharging cycles, this dataset aids in understanding the degradation patterns and operational efficiencies of batteries. The provided graphs in the Fig 3.4 collectively illustrate how battery characteristics such as voltage response and SOC progression alter with repeated use. Fig 3.4 (a) denotes the time difference between first and last value by cycle while charging, Fig 3.4 (b) denotes the Capacity vs Voltage Curve for a cell for fresh and last cycle and Fig 3.4 (c) denotes the difference between first and last charge values for both charge and discharge in percentage.

Chapter 4

Results and Discussions

Several experiments were conducted as detailed in the next part to confirm the efficacy of the above-mentioned approach. The deep learning framework Pytorch was utilized to implement the model in Python 3.9.

4.1 Evaluation Criteria

For the RUL prediction task and SOH monitoring work, separate assessment criteria were used to evaluate the model's performance. The absolute mean percentage error (MAPE) and root mean square error (RMSE) were employed for SOH monitoring. We selected Relative Error (RE) as our primary evaluation parameter for RUL prediction since it has a strong correlation with a battery's RUL. The following metrics were used to compute the results

4.1.1 MAPE

The Mean Absolute Percentage Error (MAPE) stands as a widely adopted metric within forecasting and statistical analysis for evaluating the precision of a predictive model. It delineates the average percentage deviation between anticipated and observed values. The formula for MAPE is as follows:

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{\hat{c}_i - c_i}{c_i} \right| \times 100\%$$

4.1.2 RMSE

In machine learning and statistics, the RMSE is a widely used metric to assess predictive model accuracy, particularly in regression scenarios. Computed as the square root of the mean of squared discrepancies between the actual and predicted values, this metric is an extension of the mean squared error (MSE). The formula for RMSE is:

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^n (\hat{c}_i - c_i)^2}{n}}$$

4.1.3 R squared

Regression model fit to data is assessed statistically using the R-squared (R^2) statistic. It shows what part of the variation of the dependent variable the model's independent variables can account for. The formula for R^2 is given by:

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{c}_i - c_i)^2}{\sum_{i=1}^n (c - c_i)^2}$$

4.1.4 Relative Error

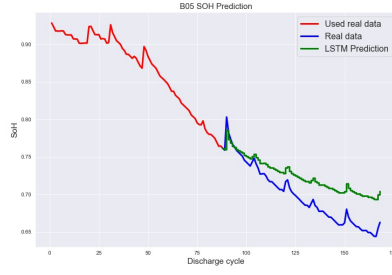
Relative Error measures the difference between predicted and actual values as a fraction or percentage of the actual value. It's a normalized metric, useful for comparing errors across different scales of data, facilitating more meaningful evaluation in various applications.

$$\text{RE} = \left| \frac{\text{RUL}_{\text{pred}} - \text{RUL}_{\text{true}}}{\text{RUL}_{\text{true}}} \right|$$

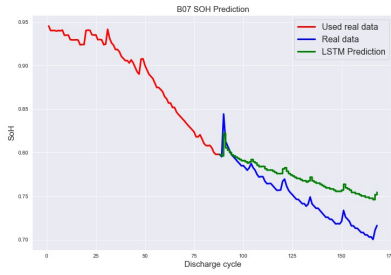
where n is the test set sample number, $\hat{c}_i$ is the capacity of the i -th cycle projected value, c_i is the actual capacity value, c is the true capacity average, RUL_{pred} is the predicted remaining life, and RUL_{true} is the actual remaining life.

4.2 Hyperparameter Optimization Results

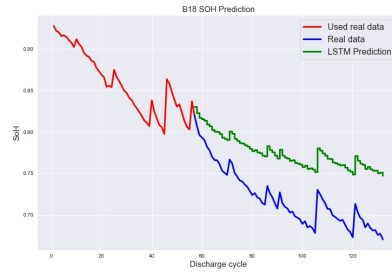
Sampler size (m), learning rate (τ), depth (l), and hidden size (s) of Transformer, regularization for learning (λ), and ratio of each task (α) are the six main parameters of the AttentiveLSTM model when utilized for RUL prediction. You can set m to be roughly 5–10% of a sequence's length. For the NASA dataset, m is fixed at 16 in our experiments. Using grid-search on the validation error, the remaining parameters were found as follows: τ is selected from $\{10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}, 10^{-2}\}$; s is selected from $\{8, 16, 32, 64\}$; l is selected from $\{1, 2, 3, 4\}$; λ is selected from $\{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}\}$; α is set from $(0, 1]$. Fig 4.1 and 4.2 show us the AttentiveLSTM based SOH predictions on NASA dataset when trained on 50% and 70% of the dataset respectively.



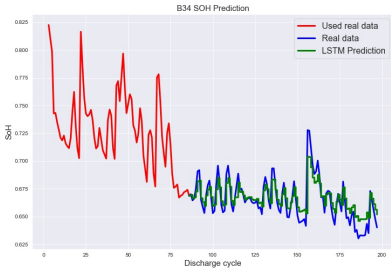
(a) Cell B0005



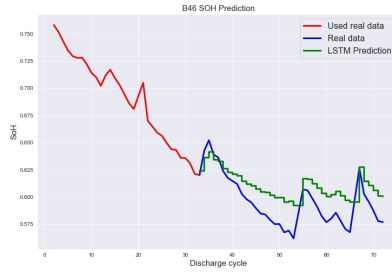
(b) Cell B0007



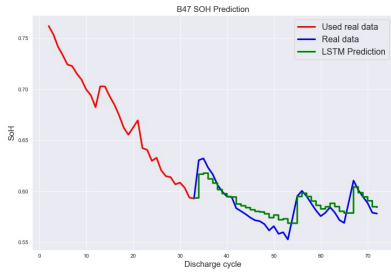
(c) Cell B0018



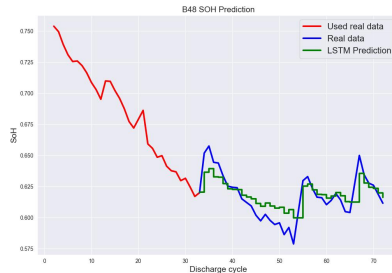
(d) Cell B0034



(e) Cell B0046

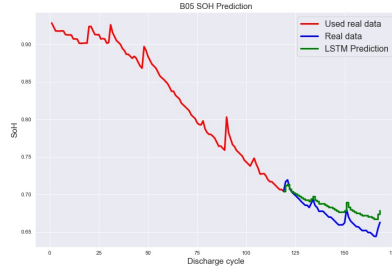


(f) Cell B0047

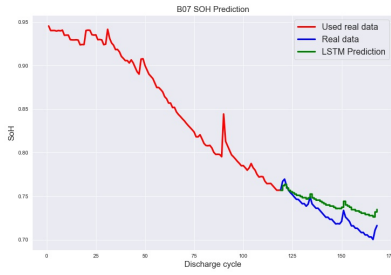


(g) Cell B0048

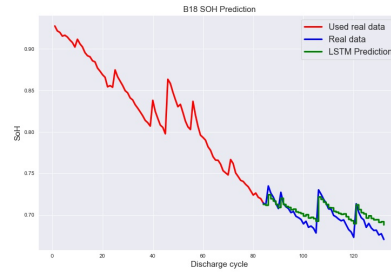
Figure 4.1: AttentiveLSTM-based SOH Prediction on NASA Dataset when trained on **50%** of the dataset



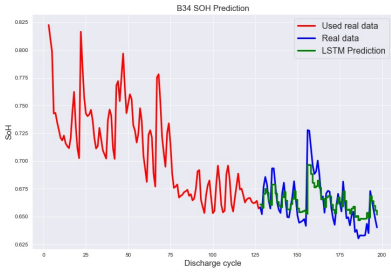
(a) Cell B0005



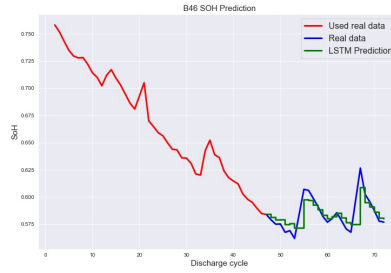
(b) Cell B0007



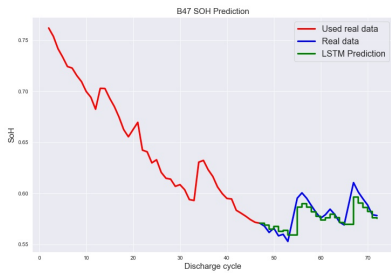
(c) Cell B0018



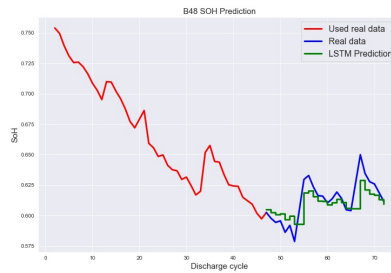
(d) Cell B0034



(e) Cell B0046



(f) Cell B0047



(g) Cell B0048

Figure 4.2: AttentiveLSTM-based SOH Prediction on NASA Dataset when trained on **70%** of the dataset

4.3 SOH Estimation Results

This section involved testing the suggested approach. As was previously indicated, there can be significant differences in the early, medium, and late stages of lithium battery deterioration. Different sets of controlled tests were run in the experiment at 50% and 70% of the dataset in order to mimic this scenario. When LSTM was used instead of other models, the performance was much better, and the datasets' MAE, RMSE, and RE scores improved as well. Tables 4.1 for the NASA dataset and 4.2 for the Oxford Degradation Dataset demonstrate this important structural enhancement brought about by the AttentiveLSTM method.

Table 4.1: Comparison of RMSE between Linear Regression and LSTM models for SOH prediction on the NASA dataset using different training data proportions (50% and 70%)

	B05		B07		B18		B33	
	50%	70%	50%	70%	50%	70%	50%	70%
Linear Regression	0.1599	0.1431	0.1190	0.1315	0.1389	0.1231	0.0303	0.0351
LSTM	0.03	0.014	0.026	0.015	0.056	0.009	0.024	0.016

	B34		B46		B47		B48	
	50%	70%	50%	70%	50%	70%	50%	70%
Linear Regression	0.0255	0.0273	0.0654	0.0385	0.1129	0.0543	0.0973	0.0470
LSTM	0.008	0.011	0.018	0.007	0.008	0.006	0.009	0.009

Table 4.2: MAE Evaluation: Linear Regression vs. LSTM for SOH Prediction on NASA Battery Data (50% vs. 70% Training Split)

	B05		B07		B18		B33	
	50%	70%	50%	70%	50%	70%	50%	70%
Regression	0.1595	0.0934	0.0777	0.1307	0.1380	0.0803	0.0258	0.0202
LSTM	0.026	0.012	0.023	0.013	0.053	0.007	0.02	0.012

	B34		B46		B47	
	50%	70%	50%	70%	50%	70%
Regression	0.0196	0.0154	0.0524	0.0207	0.0979	0.0314
LSTM	0.006	0.008	0.016	0.005	0.007	0.005

Fig. 4.1 and 4.2 illustrates the findings for AttentiveLSTM-based SOH estimation at 50% and 70% of the training dataset. The results for Cells 5, 6, 7, and 8 in Fig. 4.3 illustrate how the same findings were confirmed on the Oxford Battery Degradation

Table 4.3: AttentiveLSTM vs. Baseline Models for RUL Predictions

Cell	Metric	AttentiveLSTM	GRU	RF
5	MAE	0.00348	0.00661	0.004571
	RMSE	0.370	0.666	0.464
6	MAE	0.00366	0.00586	0.00372
	RMSE	0.540	0.724	0.564
7	MAE	0.00138	0.00118	0.00296
	RMSE	0.164	0.147	0.335
8	MAE	0.00127	0.00161	0.00316
	RMSE	0.165	0.206	0.376

datasets. This demonstrates the great degree of generalisation capabilities of the technique suggested in this research by confirming that it accurately captures the SOH decay models of various batteries.

Furthermore, a comparative test between other algorithms was carried out to offer greater proof of the efficiency and stability of the suggested approach. It was shown that the LSTM with Transformer technique greatly improved the overall model's accuracy and robustness when compared to AttentiveLSTM. The suggested AttentiveLSTM model demonstrated exceptional performance in handling time series data, including lithium battery decay data, as compared to the Gated Recurrent Unit model, an algorithm renowned for its potent time information processing capabilities.

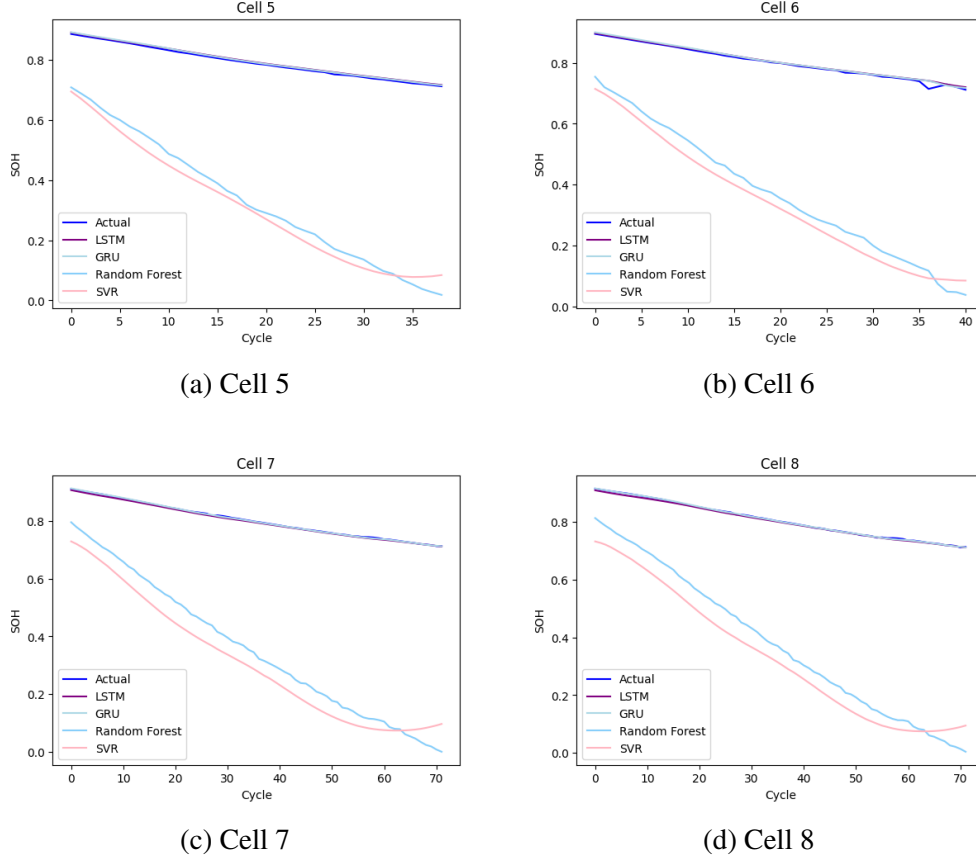


Figure 4.3: Performance of AttentiveLSTM and Baseline Models on Test Dataset of Oxford Battery Degradation

Concurrently, a comparison was conducted using the regression models, Random Forest in Table 4.3. The comparison between Fig. 4.3 (a), (b), (c), and (d) and the other three algorithms shows that the SOH evaluation curve produced by the technique suggested in this paper substantially resembles the actual curve.

4.4 RUL Prediction Results

Initially, tests were run on the NASA data set to confirm how well our approaches worked. The RE, MAE, and RMSE ratings for each approach are displayed in Table 4.4. The best outcomes are highlighted in bold. Drawing from Table 4.4 results, we deduce the following: Our models provide the best experimental outcomes across all approaches. The outcomes show that the modeling capacity sequences include valuable temporal information that our model is able to retrieve. The robustness and stability of AttentiveLSTM ensure that it consistently produces accurate predictions, irrespective of the length of a capacity sequence. A lengthy sequence may provide networks with enough information to train; on the other hand, a short sequence's representation capacity is constrained when the network receives only one feature. AttentiveLSTM per-

forms incredibly well across all parameters, with the exception of RE, which is closely correlated with a battery’s RUL. One possible explanation for this could be that the regeneration of battery charge and discharge impairs the model’s ability to learn about trends. Our models use an autoencoder to decrease noise in a raw sequence in order to improve the representation. In conclusion, our method performs better than competing strategies, indicating that it is useful in obtaining significant temporal information for the purpose of more precisely predicting a battery’s RUL.

Table 4.4: Comparison of RUL prediction performance between LSTM and AttentiveLSTM (Transformer-based) models.

	LSTM	AttentiveLSTM (Transformer-based)
RE	0.2648	0.2252
MAE	0.0829	0.0713
RMSE	0.0905	0.0802

Chapter 5

Conclusion and Future Work

5.1 Conclusion

In order to capture late-stage ageing properties of lithium batteries, the suggested methodology makes use of the novel AttentiveLSTM model, a data-driven multi-task learning algorithm that combines the powerful feature extraction capabilities of LSTM and Transformer architectures. The NASA and Oxford Battery Degradation datasets demonstrate the superior performance of the AttentiveLSTM model over other algorithms, with state-of-health (SOH) evaluation errors falling between 0.00127 and 0.00366 and remaining usable life (RUL) prediction errors of 0.2252 percent. Especially, the model shows good generalisation across different types of lithium batteries.

Additionally, RUL prediction experiments on the NASA dataset further underscore the superiority of our approach, with AttentiveLSTM consistently outperforming other methods in terms of RE, MAE, and RMSE scores. Particularly, AttentiveLSTM demonstrates stability and robustness across battery capacity sequences of varying lengths, suggesting its efficacy in extracting meaningful temporal features for accurate RUL prediction. These findings highlight the practical value of our framework in future engineering applications, emphasizing its potential to enhance battery management systems' performance and reliability.

5.2 Future Work

In the future, as advancements in battery technology emerge, there is potential to enhance the efficiency of machine learning algorithms for state-of-health (SOH) and remaining useful life (RUL) prediction. These advancements could lead to reduced evaluation times. However, it's worth noting that while LSTM models excel with larger datasets, GRU models are better suited for smaller, repetitive use cases, such as individual battery monitoring. For mass production scenarios involving similar Li-Ion batteries, LSTM models could offer insights into necessary maintenance. Moving forward, our research aims to broaden its scope by incorporating more comprehensive datasets,

addressing potential biases, and accounting for varying operational conditions like temperature and current, which significantly impact battery degradation trends. This expansion will enable more robust SOH and RUL estimations, contributing to enhanced consumer safety and commercial viability.

5.3 Task, Achievements and Possible Beneficiaries

The task at hand involves comprehensively examining data-driven methodologies for predicting the State of Health (SOH) and Remaining Useful Life (RUL) of lithium-ion batteries, particularly in the context of electric vehicles (EVs). This involves delving into various estimation methods, exploring existing research literature, identifying challenges, proposing a novel approach, and conducting experiments with relevant datasets to validate the proposed AttentiveLSTM method.

The achievements encompass a comprehensive review of existing research literature on State of Health (SOH) and Remaining Useful Life (RUL) prediction methods for lithium-ion batteries in electric vehicles (EVs). This literature review covers a wide range of techniques, including experiment-based, model-based, and machine-learning-based approaches. Building upon this review, a novel methodology termed AttentiveLSTM is introduced, which integrates LSTM with Transformer networks for simultaneous SOH and RUL prediction. This innovative approach addresses the limitations of existing methods and offers improved accuracy and efficiency. Subsequent experiments with datasets from NASA and the Oxford Battery Degradation datasets validate the proposed model. These experiments involve rigorous preprocessing of the data, structuring it into sequences, training the model, and evaluating its performance using metrics such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). Furthermore, detailed exploratory data analysis (EDA) of the datasets reveals insights into battery capacity degradation patterns over time and under different charging conditions, providing valuable information for the development of predictive models.

The research outcomes hold promise for various stakeholders within the electric vehicle (EV) ecosystem. Firstly, the electric vehicle industry stands to benefit significantly from the proposed methodologies, which can enhance battery management systems, thereby improving the reliability and efficiency of EV batteries. This advancement can pave the way for the development of safer and more sustainable electric vehicles, driving further adoption in the market. Battery manufacturers can leverage the insights garnered from this research to optimize battery design, improve longevity, and devise better predictive maintenance strategies. By reducing costs and enhancing customer satisfaction, manufacturers can strengthen their competitive edge in the market. Additionally, the research findings contribute valuable knowledge to the broader research community, serving as

a foundation for future studies and advancements in battery technology and electric vehicles. Ultimately, end users, namely EV owners, stand to benefit from longer battery lifespan, reduced maintenance costs, and an enhanced driving experience. This, in turn, can accelerate the adoption of electric vehicles, contributing to a greener and more sustainable future for transportation.

Bibliography

- [1] Z. He, M. Gao, G. Ma, Y. Liu, and S. Chen, "Online state-of-health estimation of lithium-ion batteries using dynamic bayesian networks," *Journal of Power Sources*, vol. 267, pp. 576–583, 2014.
- [2] R. Xiong, F. Sun, Z. Chen, and H. He, "A data-driven multi-scale extended kalman filtering based parameter and state estimation approach of lithium-ion polymer battery in electric vehicles," *Applied Energy*, vol. 113, pp. 463–476, 2014.
- [3] Y. Zou, X. Hu, H. Ma, and S. E. Li, "Combined state of charge and state of health estimation over lithium-ion battery cell cycle lifespan for electric vehicles," *Journal of Power Sources*, vol. 273, pp. 793–803, 2015.
- [4] Z. Guo, X. Qiu, H. Guangdong, B. Y. Liaw, and C. Zhang, "State of health estimation for lithium ion batteries based on charging curves," *Journal of Power Sources*, vol. 249, pp. 457–462, 03 2014.
- [5] X. Han, M. Ouyang, L. Lu, and J. Li, "A comparative study of commercial lithium ion battery cycle life in electric vehicle: Capacity loss estimation," *Journal of Power Sources*, vol. 268, pp. 658–669, 2014.
- [6] X. Zhang, Y. Wang, C. Liu, and Z. Chen, "A novel approach of battery pack state of health estimation using artificial intelligence optimization algorithm," *Journal of Power Sources*, vol. 376, pp. 191–199, 2018.
- [7] J. Yang, B. Xia, W. Huang, Y. Fu, and C. Mi, "Online state-of-health estimation for lithium-ion batteries using constant-voltage charging current analysis," *Applied Energy*, vol. 212, pp. 1589–1600, 2018.
- [8] I. Marri, E. Petkovski, L. Cristaldi, and M. Faifer, "Comparing machine learning strategies for soh estimation of lithium-ion batteries using a feature-based approach," *Energies*, vol. 16, no. 11, 2023.
- [9] X. Hu, S. E. Li, Z. Jia, and B. Egardt, "Enhanced sample entropy-based health management of li-ion battery for electrified vehicles," *Energy*, vol. 64, pp. 953–960, 2014.

- [10] V. Klass, M. Behm, and G. Lindbergh, "A support vector machine-based state-of-health estimation method for lithium-ion batteries under electric vehicle operation," *Journal of Power Sources*, vol. 270, pp. 262–272, 2014.
- [11] G. won You, S. Park, and D. Oh, "Real-time state-of-health estimation for electric vehicle batteries: A data-driven approach," *Applied Energy*, vol. 176, pp. 92–103, 2016.
- [12] Y. Guo, D. Yang, K. Zhao, and K. Wang, "State of health estimation for lithium-ion battery based on bi-directional long short-term memory neural network and attention mechanism," *Energy Reports*, vol. 8, pp. 208–215, 2022, 2022 International Conference on the Energy Internet and Energy Interactive Technology.
- [13] G. Cheng, X. Wang, and Y. He, "Remaining useful life and state of health prediction for lithium batteries based on empirical mode decomposition and a long and short memory neural network," *Energy*, vol. 232, p. 121022, 2021.
- [14] D. Roman, S. Saxena, V. Robu, M. G. Pecht, and D. Flynn, "Machine learning pipeline for battery state-of-health estimation," *Nature Machine Intelligence*, vol. 3, pp. 447 – 456, 2021.
- [15] N. Zhang, A. Xu, K. Wang, X. Han, W. Hong, and S. H. Hong, "Remaining useful life prediction of lithium batteries based on extended kalman particle filter," *IEEEJ Transactions on Electrical and Electronic Engineering*, vol. 16, no. 2, pp. 206–214, 2021.
- [16] X. Qiu, W. Wu, and S. Wang, "Remaining useful life prediction of lithium-ion battery based on improved cuckoo search particle filter and a novel state of charge estimation method," *Journal of Power Sources*, vol. 450, p. 227700, 2020.
- [17] W. Yan, B. Zhang, G. Zhao, S. Tang, G. Niu, and X. Wang, "A battery management system with a lebesgue-sampling-based extended kalman filter," *IEEE transactions on industrial electronics*, vol. 66, no. 4, pp. 3227–3236, 2018.
- [18] X. Li, Z. Huang, J. Tian, and Y. Tian, "State-of-charge estimation tolerant of battery aging based on a physics-based model and an adaptive cubature kalman filter," *Energy*, vol. 220, p. 119767, 2021.
- [19] M.-H. Hung, C.-H. Lin, L.-C. Lee, and C.-M. Wang, "State-of-charge and state-of-health estimation for lithium-ion batteries based on dynamic impedance technique," *Journal of Power Sources*, vol. 268, pp. 861–873, 2014.
- [20] P. Keil and A. Jossen, "Impact of dynamic driving loads and regenerative braking on the aging of lithium-ion batteries in electric vehicles," *Journal of The*

Electrochemical Society, vol. 164, no. 13, p. A3081, oct 2017. [Online]. Available: <https://dx.doi.org/10.1149/2.0801713jes>

- [21] K. A. Severson, P. M. Attia, N. Jin, N. Perkins, B. Jiang, Z. Yang, M. H. Chen, M. Aykol, P. K. Herring, D. Fraggedakis, M. Z. Bazant, S. J. Harris, W. C. Chueh, and R. D. Braatz, “Data-driven prediction of battery cycle life before capacity degradation,” *Nature Energy*, vol. 4, pp. 383 – 391, 2019.
- [22] H. Tian, P. Qin, K. Li, and Z. Zhao, “A review of the state of health for lithium-ion batteries: Research status and suggestions,” *Journal of Cleaner Production*, vol. 261, p. 120813, 2020.
- [23] H. H. Goh, Z. Lan, D. Zhang, W. Dai, T. A. Kurniawan, and K. C. Goh, “Estimation of the state of health (soh) of batteries using discrete curvature feature extraction,” *Journal of Energy Storage*, vol. 50, p. 104646, 2022.
- [24] Y. Zhou, M. Huang, Y. Chen, and Y. Tao, “A novel health indicator for on-line lithium-ion batteries remaining useful life prediction,” *Journal of Power Sources*, vol. 321, pp. 1–10, 2016.
- [25] G. Cheng, X. Wang, and Y. He, “Remaining useful life and state of health prediction for lithium batteries based on empirical mode decomposition and a long and short memory neural network,” *Energy*, vol. 232, p. 121022, 2021.
- [26] D. Chen, W. Zhang, C. Zhang, B. Sun, X. Cong, S. Wei, and J. Jiang, “A novel deep learning-based life prediction method for lithium-ion batteries with strong generalization capability under multiple cycle profiles,” *Applied Energy*, vol. 327, p. 120114, 2022.
- [27] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, “Systematic literature reviews in software engineering – a systematic literature review,” *Information and Software Technology*, vol. 51, no. 1, pp. 7–15, 2009, special Section - Most Cited Articles in 2002 and Regular Research Papers.
- [28] F.-K. Wang, Z. E. Amogne, J.-H. Chou, and C. Tseng, “Online remaining useful life prediction of lithium-ion batteries using bidirectional long short-term memory with attention mechanism,” *Energy*, vol. 254, p. 124344, 2022.
- [29] S. Zhao, C. Zhang, and Y. Wang, “Lithium-ion battery capacity and remaining useful life prediction using board learning system and long short-term memory neural network,” *Journal of Energy Storage*, vol. 52, p. 104901, 2022.
- [30] K. B. Bhargav, “Remaining useful life prediction of lithium-ion batteries using machine learning.”

BIO DATA



NAME : MANVI GUPTA

ROLL NO: 2020UEE4509

BRANCH NAME: ELECTRICAL ENGINEERING

SECTION : 1

DEGREE : BACHELOR'S IN TECHNOLOGY

SPECIALIZATION : ELECTRICAL ENGINEERING

RELATED RESEARCH WORK

1. Real Time Photovoltaic Power Forecasting using Machine Learning
2. Joint evaluation and prediction of SOH and RUL for Li-ion batteries in EV based on an AttentiveLSTM multi-task model
3. A Review of PV Forecasting using ML Techniques
4. Predictive Performance of EV Charging Behavior during COVID'19

BIO DATA



NAME : JISHNU MITTAL

ROLL NO: 2020UEE4542

BRANCH NAME: ELECTRICAL ENGINEERING

SECTION : 1

DEGREE : BACHELOR'S IN TECHNOLOGY

SPECIALIZATION : ELECTRICAL ENGINEERING

RELATED RESEARCH WORK

1. Real Time Photovoltaic Power Forecasting using Machine Learning
2. Joint evaluation and prediction of SOH and RUL for Li-ion batteries in EV based on an AttentiveLSTM multi-task model
3. A Review of PV Forecasting using ML Techniques
4. Predictive Performance of EV Charging Behavior during COVID'19

BIO DATA

NAME : ARCHIE ARYA

ROLL NO: 2020UEE4518

BRANCH NAME: ELECTRICAL ENGINEERING

SECTION : 1

DEGREE : BACHELOR'S IN TECHNOLOGY

SPECIALIZATION : ELECTRICAL ENGINEERING



RELATED RESEARCH WORK

1. Real Time Photovoltaic Power Forecasting using Machine Learning
2. Joint evaluation and prediction of SOH and RUL for Li-ion batteries in EV based on an AttentiveLSTM multi-task model
3. A Review of PV Forecasting using ML Techniques

BIO DATA

NAME : UDAY VARSHNEY

ROLL NO: 2020UEE4529

BRANCH NAME: ELECTRICAL ENGINEERING

SECTION : 1

DEGREE : BACHELOR'S IN TECHNOLOGY

SPECIALIZATION : ELECTRICAL ENGINEERING



RELATED RESEARCH WORK

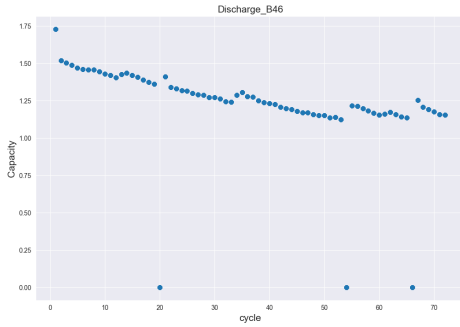
1. Real Time Photovoltaic Power Forecasting using Machine Learning
2. Joint evaluation and prediction of SOH and RUL for Li-ion batteries in EV based on an AttentiveLSTM multi-task model
3. A Review of PV Forecasting using ML Techniques

Appendices

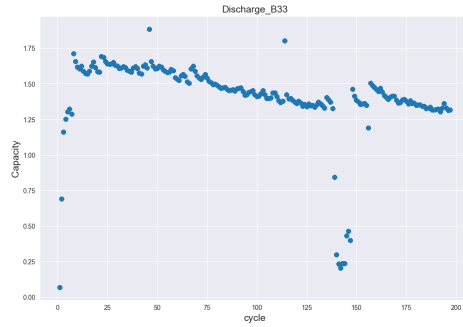
Appendix A

Exploratory Data Analysis of NASA Dataset and Results from Other Algorithms

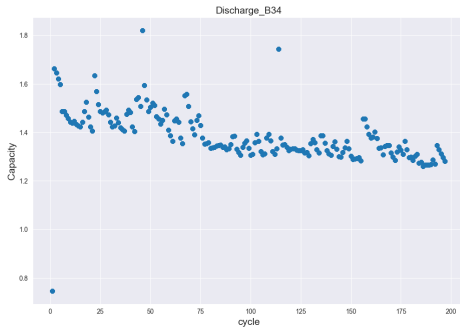
A.1 Capacity vs Cycle Discharge Curves for cells in the NASA Dataset



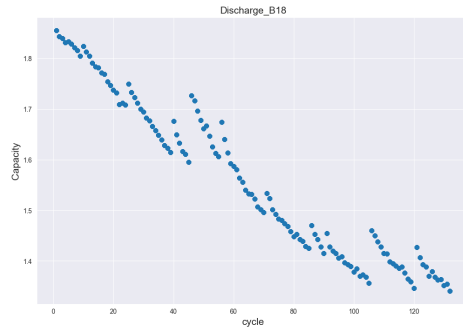
(a) Cell B0046



(b) Cell B0033



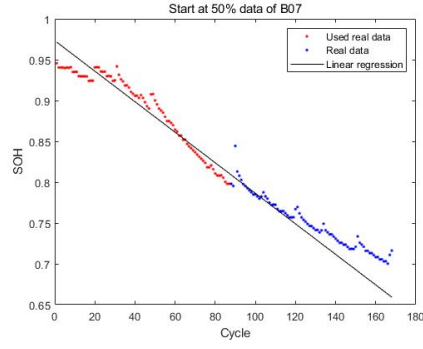
(c) Cell B0034



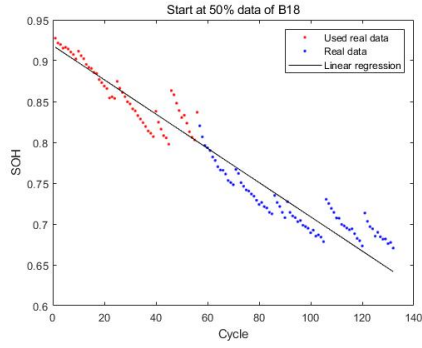
(d) Cell B0018

Figure A.1: Linear Regression-based SOH Prediction on NASA Dataset when trained on **50%** of the dataset

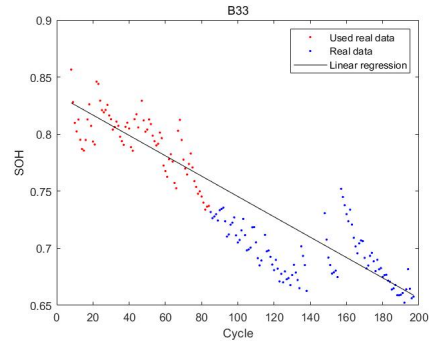
A.2 Performance of Linear Regression Model on NASA Dataset



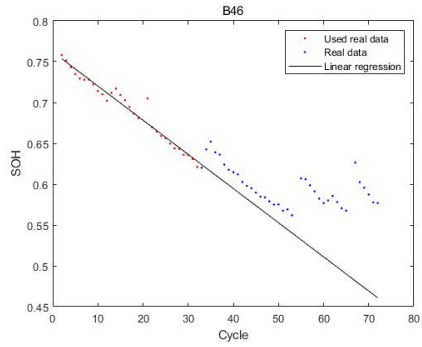
(a) Cell B0005



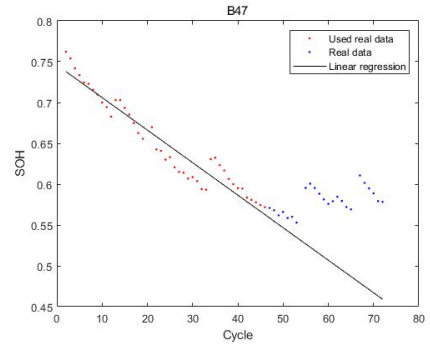
(b) Cell B0007



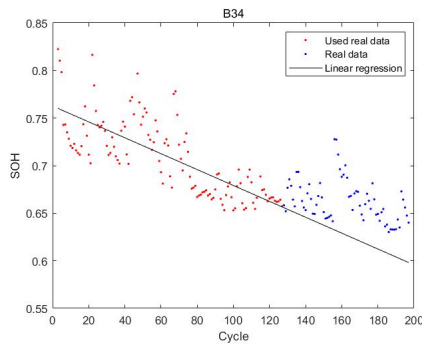
(c) Cell B0018



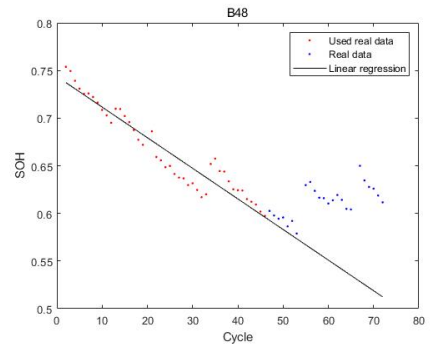
(d) Cell B0034



(e) Cell B0046

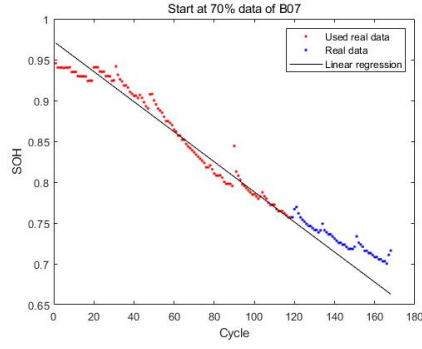


(f) Cell B0047

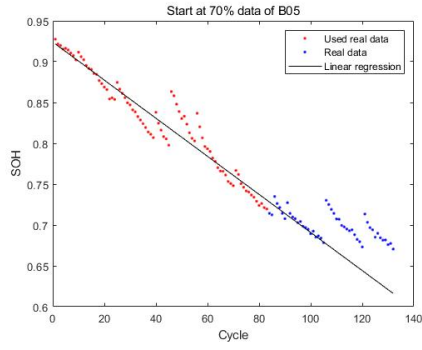


(g) Cell B0048

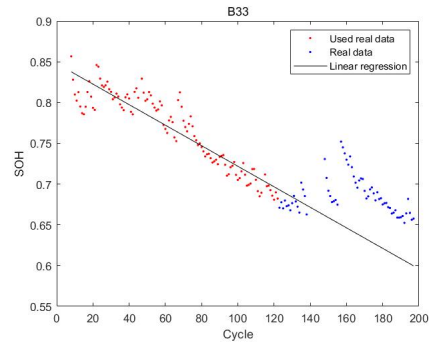
Figure A.2: Linear Regression-based SOH Prediction on NASA Dataset when trained on **50%** of the dataset



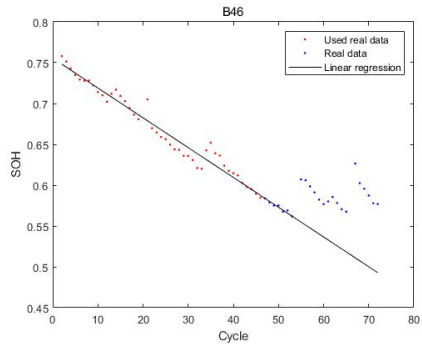
(a) Cell B0005



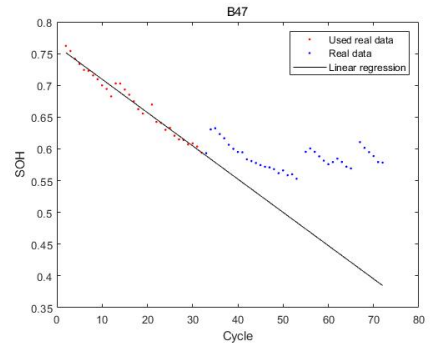
(b) Cell B0007



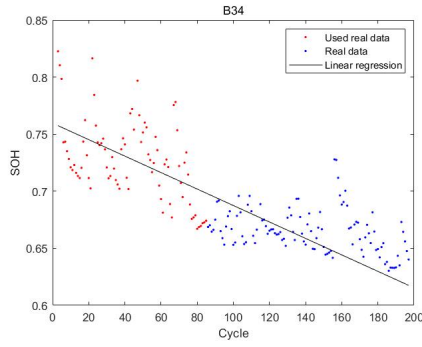
(c) Cell B0018



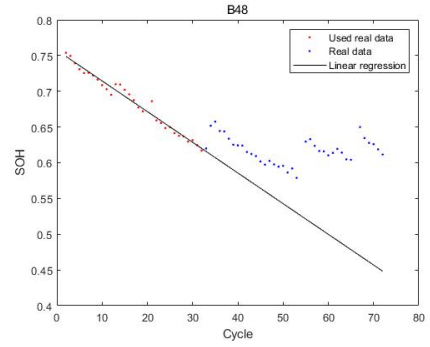
(d) Cell B0034



(e) Cell B0046



(f) Cell B0047



(g) Cell B0048

Figure A.3: Linear Regression-based SOH Prediction on NASA Dataset when trained on **70%** of the dataset

Appendix B

Code Attachments

B.1 Python Code for AttentiveLSTM Algorithm

The following code is for RUL Prediction and SOH estimation for the NASA Dataset where we have used the Transformer method for RUL and the LSTM method for SOH, for a novel AttentiveLSTM method.

```
1 # Function to convert string representation of time to datetime
  object
2 def convert_to_datetime(time_str):
3     """
4     This function takes a string representing time in HHMMSS format and
      converts it to a datetime object.
5
6     Args:
7         time_str: String representing time in HHMMSS format (e.g.,
          "123456").
8
9     Returns:
10        A datetime object representing the given time.
11    """
12    year, month, day, hour, minute, second = int(time_str[0]), int(
        time_str[1]), int(time_str[2]), int(time_str[3]), int(time_str[4]),
        int(time_str[5])
13    return datetime(year=year, month=month, day=day, hour=hour, minute=
        minute, second=second)
14
15 # Function to load .mat data
16 def load_mat(matfile):
17     """
18     This function loads data from a .mat file and processes it.
19
20     Args:
21         matfile: Path to the .mat file.
22
23     Returns:
24        A list containing dictionaries representing the loaded data.
25    """
26    data = scipy.io.loadmat(matfile)
27    filename = matfile.split("/")[-1].split(".")[0]
28    col = data[filename]
29    col = col[0][0][0][0]
30    size = col.shape[0]
```

```

31
32 data_list = []
33 for i in range(size):
34     k = list(col[i][3][0].dtype.fields.keys())
35     d1, d2 = {}, {}
36     if str(col[i][0][0]) != 'impedance':
37         for j in range(len(k)):
38             t = col[i][3][0][0][j][0];
39             l = [t[m] for m in range(len(t))]
40             d2[k[j]] = l
41             d1['type'], d1['temp'], d1['time'], d1['data'] = str(col[i]
42 ][0][0]), int(col[i][1][0]), str(convert_to_datetime(col[i][2][0])
43 ), d2
44             data_list.append(d1)
45
46 return data_list
47
48 # Convert input_data to DataFrame
49 def convert_to_dataframe(input_data, mat, CELL_SIZE):
50     """
51     This function converts input data to a DataFrame and saves it to
52     a CSV file.
53
54     Args:
55         input_data: Input data containing charging and discharging
56         capacities.
57         mat: MATLAB data containing cell information.
58         CELL_SIZE: List containing cell sizes.
59
60     Returns:
61         DataFrame: Pandas DataFrame containing cycle, charging
62         capacity, and discharging capacity.
63     """
64     data_dict = {'Cycle': [], 'ChargingCapacity': [], '
65 DischargingCapacity': []}
66
67     for i in range(0, 8):
68         cell_num = "Cell{}".format(i + 1)
69         for j in range(0, CELL_SIZE[i]):
70             cyc_num = "cyc{:04d}".format(j * 100)
71             try:
72                 charging_cap = mat[cell_num][0][cyc_num][0][0][ "C1ch"
73 ][0][0][ 'q' ][0][ -1 ][0]
74             except ValueError:
75                 charging_cap = float("NaN")
76
77             try:
78                 discharging_cap = mat[cell_num][0][cyc_num][0][0][ "
79 C1dc" ][0][0][ 'q' ][0][ -1 ][0]
80             except ValueError:
81                 discharging_cap = float("NaN")
82
83             data_dict['Cycle'].append(j)
84             data_dict['ChargingCapacity'].append(charging_cap)
85             data_dict['DischargingCapacity'].append(discharging_cap)
86
87     df = pd.DataFrame(data_dict)
88     df.to_csv('battery_data.csv', index=False)

```

```

81     return df
82
83 # Data preprocessing and visualization
84 def preprocess_and_visualize(input_data , label_sc , train_x , train_y ,
85     test_x , test_y):
86     """
87     This function preprocesses the data , performs normalization ,
88     windowing, and visualization.
89
90     Args:
91     input_data: Input data for preprocessing.
92     label_sc: Label scaler for normalization.
93     train_x: Training input data.
94     train_y: Training labels.
95     test_x: Testing input data.
96     test_y: Testing labels.
97     """
98     for i in range(0 , 8):
99         df_input = pd.DataFrame(input_data[i]).transpose()
100         df_input = df_input.rename(columns={0: "cycle", 1: "q_value"
101     })
102         df_input['interpolate_time'] = df_input['q_value'].
103         interpolate(option='time') # Time interpolate preprocessing of
104         cells 1 to 8
105         df_input['soh'] = df_input['interpolate_time'] / 740
106
107         plt.figure(figsize=(12, 6))
108         plt.subplot(1, 2, 1)
109         plt.plot(df_input['cycle'], df_input['q_value'], marker='o',
110     linestyle='-')
111         plt.xlabel('Cycle')
112         plt.ylabel('Capacity (q_value)')
113         plt.title(f'Capacity vs. Cycles for Cell {i+1}')
114
115         plt.subplot(1, 2, 2)
116         plt.plot(df_input['cycle'], df_input['soh'], marker='o',
117     linestyle='-')
118         plt.xlabel('Cycle')
119         plt.ylabel('SOH')
120         plt.title(f'SOH vs. Cycles for Cell {i+1}')
121
122         plt.tight_layout()
123         plt.show()
124
125         df_input = df_input.drop(["cycle", "q_value", "
126     interpolate_time"], axis=1)
127
128         sc = MinMaxScaler()
129         label_sc.fit(df_input.iloc[:, 0].values.reshape(-1, 1)) #
130         MinMaxScaler preprocessing of cells 1 to 8
131
132         data = sc.fit_transform(df_input.values)
133
134         lookback = 10
135         inputs = np.zeros((len(data) - lookback , lookback , df_input.
136     shape[1]))
137         labels = np.zeros(len(data) - lookback)

```

```

129         for j in range(lookback, len(data)):
130             inputs[j - lookback] = data[j - lookback:j]
131             labels[j - lookback] = data[j, 0]
132
133         inputs = inputs.reshape(-1, lookback, df_input.shape[1])
134         labels = labels.reshape(-1, 1)
135
136         if i < 4:
137             if len(train_x) == 0:
138                 train_x = inputs[:]
139                 train_y = labels[:]
140             else:
141                 train_x = np.concatenate((train_x, inputs[:]))
142                 train_y = np.concatenate((train_y, labels[:]))
143         else:
144             test_x[i - 4] = inputs
145             test_y[i - 4] = labels
146
147     # Model training
148     def train_model(train_loader, learn_rate, model_type="GRU"):
149         """
150         This function trains the GRU or LSTM model.
151
152         Args:
153             train_loader: DataLoader for training data.
154             learn_rate: Learning rate for training.
155             model_type: Type of model to train (GRU or LSTM).
156
157         Returns:
158             model: Trained GRU or LSTM model.
159         """
160         input_dim = next(iter(train_loader))[0].shape[2]
161         output_dim = 1
162         n_layers = 2
163         hidden_dim = 256
164         EPOCHS = 100
165
166         if model_type == "GRU":
167             model = GRUNet(input_dim, hidden_dim, output_dim, n_layers)
168         else:
169             model = LSTMNet(input_dim, hidden_dim, output_dim, n_layers)
170
171         model.to(device)
172
173         criterion = nn.MSELoss()
174         optimizer = torch.optim.Adam(model.parameters(), lr=learn_rate)
175
176         model.train()
177         epoch_times = []
178         for epoch in range(1, EPOCHS + 1):
179             start_time = time.process_time()
180             h = model.init_hidden(batch_size)
181             avg_loss = 0.
182             counter = 0
183             for x, label in train_loader:
184                 counter += 1
185                 if model_type == "GRU":
186                     h = h.data

```

```

187         else :
188             h = tuple([e.data for e in h])
189             model.zero_grad()
190
191             out, h = model(x.to(device).float(), h)
192             loss = criterion(out, label.to(device).float())
193             loss.backward()
194             optimizer.step()
195             avg_loss += loss.item()
196
197             current_time = time.process_time()
198             epoch_times.append(current_time - start_time)
199
200         return model
201
202 # Model evaluation
203 def evaluate_model(model, test_x, test_y, label_sc):
204     """
205     This function evaluates the trained model.
206
207     Args:
208         model: Trained model to evaluate.
209         test_x: Testing input data.
210         test_y: Testing labels.
211         label_sc: Label scaler for inverse transformation.
212
213     Returns:
214         MAE: Mean absolute error.
215         RMSE: Root mean squared error.
216     """
217     model.eval()
218     outputs = []
219     targets = []
220
221     inp = torch.from_numpy(np.array(test_x))
222     labs = torch.from_numpy(np.array(test_y))
223     h = model.init_hidden(inp.shape[0])
224     out, h = model(inp.to(device).float(), h)
225     outputs.append(label_sc.inverse_transform(out.cpu().detach().
226 numpy()).reshape(-1))
227     targets.append(label_sc.inverse_transform(labs.numpy()).reshape
228 (-1))
229
230     MAE = 0
231     RMSE = 0
232     for i in range(len(outputs)):
233         MAE += np.mean(abs(outputs[i] - targets[i])) / len(outputs)
234         RMSE += np.mean((outputs[i] - targets[i]) ** 2) / len(outputs)
235
236     RMSE = math.sqrt(RMSE)
237
238     return MAE, RMSE
239
240 # Save models
241 def save_models(gru_model, lstm_model):
242     """
243     This function saves the trained GRU and LSTM models.

```

```

242     Args:
243         gru_model: Trained GRU model.
244         lstm_model: Trained LSTM model.
245     """
246     torch.save(gru_model.state_dict(), "./saved_models/gru_model.pt")
247     torch.save(lstm_model.state_dict(), "./saved_models/lstm_model.pt")
248
249 # Plot evaluation results
250 def plot_evaluation_results(outputs, targets):
251     """
252     This function plots the evaluation results.
253
254     Args:
255         outputs: Predicted outputs.
256         targets: Actual targets.
257     """
258     plt.figure(figsize=(14, 10))
259     plt.plot(outputs[0][-100:], "-o", color="g", label="Predicted")
260     plt.plot(targets[0][-100:], color="b", label="Actual")
261     plt.xlabel('Cycle')
262     plt.ylabel('SOH')
263     plt.legend()
264     plt.show()
265
266 ## RUL Prediction
267 # Function to get capacity data from Battery
268 def get_battery_capacity(battery):
269     """
270     This function extracts capacity data from the battery data.
271
272     Args:
273         battery: List containing battery data.
274
275     Returns:
276         A tuple containing lists of discharge cycles and corresponding
277         capacities.
278     """
279     cycle, capacity = [], []
280     i = 1
281     for bat in battery:
282         if bat['type'] == 'discharge':
283             capacity.append(bat['data']['Capacity'][0])
284             cycle.append(i)
285             i += 1
286     return [cycle, capacity]
287
288 # Function to get the charge or discharge data of a battery
289 def get_battery_values(battery, type='charge'):
290     """
291     This function extracts charge or discharge data of a battery from
292     the battery data.
293
294     Args:
295         battery: List containing battery data.
296         type: Type of data to extract (default is 'charge').
297
298     Returns:

```

```

297     A list of dictionaries containing the requested battery data.
298     """
299     data = []
300     for bat in battery:
301         if bat['type'] == type:
302             data.append(bat['data'])
303     return data
304
305 # Function to build sequences for training
306 def build_sequences(text, window_size):
307     """
308     This function builds sequences from the input text data for
309     training.
310
311     Args:
312         text: List of capacity data.
313         window_size: Size of the sliding window for building sequences.
314
315     Returns:
316         Arrays containing input-output pairs of sequences.
317     """
318     x, y = [], []
319     for i in range(len(text) - window_size):
320         sequence = text[i:i+window_size]
321         target = text[i+1:i+1+window_size]
322
323         x.append(sequence)
324         y.append(target)
325
326     return np.array(x), np.array(y)
327
328 # Function to split dataset into training and testing data
329 def split_dataset(data_sequence, train_ratio=0.0, capacity_threshold
330                  =0.0):
331     """
332     This function splits the dataset into training and testing data.
333
334     Args:
335         data_sequence: Sequence of data to split.
336         train_ratio: Ratio of data to use for training (default is 0.0)
337         .
338         capacity_threshold: Threshold capacity value for splitting (
339         default is 0.0).
340
341     Returns:
342         Lists containing training and testing data.
343     """
344     if capacity_threshold > 0:
345         max_capacity = max(data_sequence)
346         capacity = max_capacity * capacity_threshold
347         point = [i for i in range(len(data_sequence)) if data_sequence[
348             i] < capacity]
349     else:
350         point = int(train_ratio + 1)
351         if 0 < train_ratio <= 1:
352             point = int(len(data_sequence) * train_ratio)
353     train_data, test_data = data_sequence[:point], data_sequence[point
354 : ]

```

```

349     return train_data , test_data
350
351 # Function for leave-one-out evaluation
352 def get_train_test(data_dict , name, window_size=8):
353     """
354     This function prepares training and testing data for leave-one-out
355     evaluation.
356
357     Args:
358         data_dict: Dictionary containing battery data.
359         name: Name of the battery to leave out for testing.
360         window_size: Size of the sliding window for building sequences
361                     (default is 8).
362
363     Returns:
364         Training data , testing data , and lists of capacity data.
365     """
366     data_sequence = data_dict[name][1]
367     train_data , test_data = data_sequence[:window_size+1],
368                               data_sequence[window_size+1:]
369     train_x , train_y = build_sequences(text=train_data , window_size=
370                                       window_size)
371     for k, v in data_dict.items():
372         if k != name:
373             data_x , data_y = build_sequences(text=v[1], window_size=
374                                               window_size)
375             train_x , train_y = np.r_[train_x , data_x], np.r_[train_y ,
376                                                                 data_y]
377
378     return train_x , train_y , list(train_data) , list(test_data)
379
380 # Function to calculate relative error
381 def relative_error(y_test , y_predict , threshold):
382     """
383     This function calculates the relative error between test and
384     predicted values.
385
386     Args:
387         y_test: List of actual test values.
388         y_predict: List of predicted values.
389         threshold: Threshold value for relative error calculation.
390
391     Returns:
392         Relative error between test and predicted values.
393     """
394     true_re , pred_re = len(y_test) , 0
395     for i in range(len(y_test)-1):
396         if y_test[i] <= threshold >= y_test[i+1]:
397             true_re = i - 1
398             break
399     for i in range(len(y_predict)-1):
400         if y_predict[i] <= threshold:
401             pred_re = i - 1
402             break
403     return abs(true_re - pred_re)/true_re
404
405 # Function for model evaluation
406 def evaluation(y_test , y_predict):

```



```

400 """
401 This function evaluates the model performance using root mean
    squared error.
402
403 Args:
404     y_test: List of actual test values.
405     y_predict: List of predicted values.
406
407 Returns:
408     Root mean squared error (RMSE) between test and predicted
    values.
409 """
410 mse = mean_squared_error(y_test, y_predict)
411 rmse = sqrt(mean_squared_error(y_test, y_predict))
412 return rmse
413
414 # Function to set random seed for reproducibility
415 def setup_seed(seed):
416     """
417     This function sets random seed for reproducibility.
418
419     Args:
420         seed: Random seed value.
421     """
422     np.random.seed(seed) # Numpy module.
423     random.seed(seed) # Python random module.
424     os.environ['PYTHONHASHSEED'] = str(seed)
425     torch.manual_seed(seed)
426     if torch.cuda.is_available():
427         torch.cuda.manual_seed(seed)
428         torch.cuda.manual_seed_all(seed)
429         torch.backends.cudnn.benchmark = False
430         torch.backends.cudnn.deterministic = True
431
432 # Autoencoder class definition
433 class Autoencoder(nn.Module):
434     """
435     Autoencoder neural network model.
436     """
437     def __init__(self, input_size=16, hidden_dim=8, noise_level=0.01):
438         super(Autoencoder, self).__init__()
439         self.input_size, self.hidden_dim, self.noise_level = input_size
440         , hidden_dim, noise_level
441         self.fc1 = nn.Linear(self.input_size, self.hidden_dim)
442         self.fc2 = nn.Linear(self.hidden_dim, self.input_size)
443
444     def encoder(self, x):
445         x = self.fc1(x)
446         h1 = F.relu(x)
447         return h1
448
449     def mask(self, x):
450         corrupted_x = x + self.noise_level * torch.randn_like(x)
451         return corrupted_x
452
453     def decoder(self, x):
454         h2 = self.fc2(x)
455         return h2

```

```

455
456     def forward(self , x):
457         out = self.mask(x)
458         encode = self.encoder(out)
459         decode = self.decoder(encode)
460         return encode , decode
461
462 # PositionalEncoding class definition
463 class PositionalEncoding(nn.Module):
464     """
465     Positional encoding for transformer model.
466     """
467     def __init__(self , d_model , dropout=0.0 , max_len=16):
468         super(PositionalEncoding , self).__init__()
469
470         pe = torch.zeros(max_len , d_model)
471         position = torch.arange(0 , max_len , dtype=torch.float).
472         unsqueeze(1)
473
474         div_term = torch.exp(torch.arange(0 , d_model , 2).float() * (-
475         math.log(10000.0) / d_model))
476
477         pe[:, 0::2] = torch.sin(position * div_term)
478         pe[:, 1::2] = torch.cos(position * div_term)
479
480         pe = pe.unsqueeze(0).transpose(0 , 1)
481
482         self.register_buffer('pe' , pe)
483
484     def forward(self , x):
485         x = x + self.pe[:x.size(1) , :].squeeze(1)
486         return x
487
488 # Net class definition
489 class Net(nn.Module):
490     """
491     Transformer neural network model.
492     """
493     def __init__(self , feature_size=16 , hidden_dim=32 , num_layers=1 ,
494     nhead=8 , dropout=0.0 , noise_level=0.01):
495         super(Net , self).__init__()
496         self.auto_hidden = int(feature_size / 2)
497         input_size = self.auto_hidden
498         self.pos = PositionalEncoding(d_model=input_size , max_len=
499         input_size)
500         encoder_layers = nn.TransformerEncoderLayer(d_model=input_size ,
501         nhead=nhead , dim_feedforward=hidden_dim , dropout=dropout)
502         self.cell = nn.TransformerEncoder(encoder_layers , num_layers=
503         num_layers)
504         self.linear = nn.Linear(input_size , 1)
505         self.autoencoder = Autoencoder(input_size=feature_size ,
506         hidden_dim=self.auto_hidden , noise_level=noise_level)
507
508     def forward(self , x):
509         batch_size , feature_num , feature_size = x.shape
510         encode , decode = self.autoencoder(x.reshape(batch_size , -1))
511         out = encode.reshape(batch_size , -1 , self.auto_hidden)
512         out = self.pos(out)

```

```

506         out = out.reshape(1, batch_size, -1)
507         out = self.cell(out)
508         out = out.reshape(batch_size, -1)
509         out = self.linear(out)
510
511         return out, decode
512
513 # Function to train the model
514 def train(lr=0.01, feature_size=8, hidden_dim=32, num_layers=1, nhead
515         =8, weight_decay=0.0, EPOCH=1000, seed=0,
516         alpha=0.0, noise_level=0.0, dropout=0.0, metric='re',
517         is_load_weights=True):
518     """
519     This function trains the transformer model.
520
521     Args:
522     lr: Learning rate for optimization.
523     feature_size: Size of the features.
524     hidden_dim: Dimensionality of the transformer model.
525     num_layers: Number of transformer layers.
526     nhead: Number of attention heads.
527     weight_decay: Weight decay for optimization.
528     EPOCH: Number of training epochs.
529     seed: Random seed value.
530     alpha: Weight for reconstruction loss.
531     noise_level: Level of noise for autoencoder.
532     dropout: Dropout rate.
533     metric: Evaluation metric (default is 're').
534     is_load_weights: Whether to load pre-trained weights.
535
536     Returns:
537     Lists containing evaluation scores and predicted results.
538     """
539     score_list, result_list = [], []
540     setup_seed(seed)
541     for i in range(4):
542         name = Battery_list[i]
543         window_size = feature_size
544         train_x, train_y, train_data, test_data = get_train_test(
545             Battery, name, window_size)
546         train_size = len(train_x)
547
548         model = Net(feature_size=feature_size, hidden_dim=hidden_dim,
549                     num_layers=num_layers, nhead=nhead, dropout=dropout,
550                     noise_level=noise_level)
551         model = model.to(device)
552         optimizer = torch.optim.Adam(model.parameters(), lr=lr,
553                                       weight_decay=weight_decay)
554         criterion = nn.MSELoss()
555
556         if is_load_weights:
557             if torch.__version__.split('+')[0] >= '1.6.0':
558                 model.load_state_dict(torch.load('initial_weights /
559 model_NASA.pth'))
560             else:
561                 model.load_state_dict(torch.load('initial_weights /
562 model_NASA_1.5.0.pth'))

```

```

557     test_x = train_data.copy()
558     loss_list, y_ = [0], []
559     rmse, re = 1, 1
560     score_, score = [1], [1]
561     for epoch in range(EPOCH):
562         X = np.reshape(train_x / Rated_Capacity, (-1, 1, feature_size)
563         ).astype(np.float32)
564         y = np.reshape(train_y[:, -1] / Rated_Capacity, (-1, 1)).astype(
565         np.float32)
566         X, y = torch.from_numpy(X).to(device), torch.from_numpy(y).
567         to(device)
568         output, decode = model(X)
569         output = output.reshape(-1, 1)
570         loss = criterion(output, y) + alpha * criterion(decode, X.
571         reshape(-1, feature_size))
572         optimizer.zero_grad()
573         loss.backward()
574         optimizer.step()
575
576         if (epoch + 1) % 10 == 0:
577             test_x = train_data.copy()
578             point_list = []
579             while (len(test_x) - len(train_data)) < len(test_data):
580                 x = np.reshape(np.array(test_x[-feature_size:])/
581                 Rated_Capacity, (-1, 1, feature_size)).astype(np.float32)
582                 x = torch.from_numpy(x).to(device)
583                 pred, _ = model(x)
584                 next_point = pred.data.cpu().numpy()[0, 0] *
585                 Rated_Capacity
586                 test_x.append(next_point)
587                 point_list.append(next_point)
588             y_.append(point_list)
589
590             loss_list.append(loss)
591             rmse = evaluation(y_test=test_data, y_predict=y_[-1])
592             re = relative_error(y_test=test_data, y_predict=y_[-1],
593             threshold=Rated_Capacity*0.7)
594             if metric == 're':
595                 score = [re]
596             elif metric == 'rmse':
597                 score = [rmse]
598             else:
599                 score = [re, rmse]
600             if (loss < 1e-3) and (score_[0] < score[0]):
601                 break
602             score_ = score.copy()
603
604             score_list.append(score_)
605             result_list.append(y_[-1])
606     return score_list, result_list
607
608 # Function to perform grid search for hyperparameters
609 def grid_search():
610     """
611     This function performs a grid search to determine optimal
612     hyperparameters.
613     """
614     states = {}

```

```

607 for lr in [1e-3, 1e-2]:
608     for num_layers in [1, 2]:
609         for hidden_dim in [16, 32]:
610             for alpha in [1e-5, 1e-4]:
611                 show_str = 'lr={}, num_layers={}, hidden_dim={},
alpha={} '.format(lr, num_layers, hidden_dim, alpha)
612                 print(show_str)
613                 SCORE = []
614                 for seed in range(5):
615                     print('seed:{}'.format(seed))
616                     score_list, _ = train(lr=lr, feature_size=
feature_size, hidden_dim=hidden_dim, num_layers=num_layers, nhead=
nhead,
617                                     weight_decay=weight_decay
, EPOCH=EPOCH, seed=seed, dropout=dropout, alpha=alpha,
618                                     noise_level=noise_level,
metric=metric, is_load_weights=is_load_weights)
619                     print(np.array(score_list))
620                     print(metric + ' : {:<6.4f} '.format(np.mean(np.
array(score_list))))
621                     for s in score_list:
622                         SCORE.append(s)
623
624                     print(metric + ' mean: {:<6.4f} '.format(np.mean(np.
array(SCORE))))
625                     states[show_str] = np.mean(np.array(SCORE))
626
627
628 min_key = min(states, key = states.get)
629 print('optimal parameters: {}, result: {}'.format(min_key, states[
min_key]))
630 # grid_search()

```